

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

KHOA CÔNG NGHỆ THÔNG TIN 1



**DỰ ĐOÁN GIÁ XE Ô TÔ CŨ CÓ XÉT ĐẾN TÌNH
TRẠNG HƯ HỎNG NGOẠI THẤT BẰNG
MACHINE LEARNING VÀ COMPUTER VISION**

Môn học: Thực tập cơ sở

Nguyễn Đức Sinh B23DCKH098

Giảng viên hướng dẫn: TS. Nguyễn Xuân Đức

Hà Nội – 2026

MỤC LỤC

1. Tổng quan đề tài.....	4
1.1. Giới thiệu bài toán định giá xe cũ	4
1.2. Lý do chọn đề tài.....	4
1.3. Mục tiêu dự án.....	5
1.4. Phạm vi.....	6
2. Cơ sở lý thuyết	7
2.1. Bài toán hồi quy trong Machine Learning	7
2.2. Random Forest, Linear Regression, XGBoost.....	7
2.2.1 Random Forest	7
2.2.1.1. Khái niệm	7
2.2.1.2. Cách thức hoạt động.....	7
2.2.1.3. Ưu và nhược điểm của thuật toán	8
2.2.1.4. Vai trò của Random Forest trong dự án.....	8
2.2.2 Linear Regression.....	9
2.2.2.1. Khái niệm	9
2.2.2.2. Cách thức hoạt động.....	9
2.2.2.3. Ưu và nhược điểm của thuật toán	10
2.2.2.4. Vai trò của Linear Regression trong dự án	10
2.2.3. XGBoost.....	10
2.2.3.1. Khái niệm	10
2.2.3.2. Cách thức hoạt động.....	11
2.2.3.3. Ưu và nhược điểm của thuật toán	11
2.2.3.4. Vai trò của XGBoost trong dự án	12
2.3. YOLO cho object detection	12
2.4. CNN/ResNet18 cho phân loại ảnh	16
2.5. Các metric: RMSE, MAE, R^2 , mAP, accuracy	17
2.5.1. Metrics Cho Bài Toán Dự Đoán Giá Xe.....	17
2.5.2 Metrics Cho Bài Toán Object Detection Bằng YOLO	17
2.5.3. Metrics cho bài toán phân loại mức độ hư hỏng.....	18
2.5.4. Vai trò của metrics trong dự án.....	19

3. Dataset và tiền xử lý	19
3.1. Dataset giá xe cũ dạng bảng.....	19
3.2. Dataset phát hiện hư hỏng xe	20
3.3. Dataset phân loại mức độ hư hỏng.....	21
3.4. Nhận xét về dữ liệu	22
4. Phương pháp xây dựng mô hình	22
4.1. Mô hình dự đoán giá từ dữ liệu bảng	22
4.2. Mô hình phát hiện hư hỏng bằng YOLOv8	23
4.3. Mô hình phân loại mức độ hư hỏng bằng ResNet18	24
4.4. Cơ chế điều chỉnh giá sau hư hỏng	24
5. Thiết kế hệ thống.....	25
5.1. Kiến trúc tổng thể.....	25
5.2. Luồng xử lý	27
5.3. Các module chính.....	29
6. Kết quả thực nghiệm	31
6.1. Kết quả mô hình dự đoán giá từ dữ liệu bảng.....	31
6.2. Kết quả thực nghiệm mô hình YOLO.....	32
6.2.1. Kết quả huấn luyện trên các cấu hình nền tảng	32
6.2.2. Tác động của dataset merge_data	32
6.2.3. Quyết định lựa chọn mô hình cốt lõi merge_data.....	33
6.2.4. Nhận xét so sánh với Faster R-CNN.....	38
6.3. Kết quả CNN	39
7. Đánh giá, hạn chế và hướng phát triển.....	41
7.1. Hạn chế.....	41
7.2. Hướng phát triển	42
8. Kết luận	43
9. Tài liệu tham khảo.....	44

LỜI MỞ ĐẦU

Lời đầu tiên, em xin gửi lời cảm ơn chân thành đến Học viện Công nghệ Bưu chính Viễn thông đã tạo điều kiện cho em được tiếp cận với môn học Thực tập cơ sở – một học phần quan trọng giúp em bước đầu làm quen với quy trình xây dựng một sản phẩm công nghệ thực tế.

Đặc biệt, em xin gửi lời cảm ơn sâu sắc nhất tới thầy **Nguyễn Xuân Đức**. Trong suốt quá trình thực hiện học phần này, thầy đã tận tình hướng dẫn, định hướng chuyên môn và cung cấp cho em những kiến thức nền tảng quý báu để em có thể hoàn thành đề tài dự án cá nhân của mình.

Hiện nay, cùng với sự phát triển mạnh mẽ của thị trường xe hơi, việc xác định giá trị thực của một chiếc xe đã qua sử dụng trở thành nhu cầu thiết yếu của cả người mua lẫn người bán. Nhận thấy tiềm năng và tính ứng dụng thực tiễn của trí tuệ nhân tạo trong việc giải quyết bài toán này, em đã quyết định lựa chọn đề tài: **“Dự đoán giá xe ô tô cũ có xét đến tình trạng hư hỏng ngoại thất bằng Machine Learning và Computer Vision”**. Đề tài hướng đến việc kết hợp dữ liệu dạng bảng với dữ liệu ảnh ngoại thất để hỗ trợ ước lượng giá xe cũ một cách trực quan và gần với tình huống thực tế hơn.

Do đây là lần đầu tiên trực tiếp triển khai một dự án Machine Learning và Computer Vision từ bước thu thập dữ liệu, tiền xử lý, huấn luyện mô hình cho đến tích hợp vào ứng dụng demo, mặc dù đã nỗ lực hết mình nhưng bài báo cáo chắc chắn không tránh khỏi những thiếu sót về mặt kỹ thuật hay cách trình bày. Em rất mong nhận được những ý kiến đóng góp, nhận xét từ thầy để em có thể rút kinh nghiệm và hoàn thiện tư duy lập trình cũng như kiến thức chuyên môn của mình trong tương lai.

Cuối cùng, em xin kính chúc thầy thật nhiều sức khỏe, hạnh phúc và gặt hái được nhiều thành công hơn nữa trong sự nghiệp giảng dạy.

Em xin chân thành cảm ơn!

1. Tổng quan đề tài

1.1. Giới thiệu bài toán định giá xe cũ

Trong kỷ nguyên công nghệ 4.0, sự bùng nổ của dữ liệu và sức mạnh tính toán đã đưa Học máy (Machine Learning) trở thành công cụ cốt lõi trong việc ra quyết định ở nhiều lĩnh vực. Trong đó, thị trường ô tô đã qua sử dụng (Used Cars) là một thị trường có tính biến động cao và chịu ảnh hưởng bởi rất nhiều yếu tố đan xen.

Việc xác định giá trị thực của một chiếc xe cũ từ trước đến nay thường dựa vào kinh nghiệm cá nhân của các chuyên gia hoặc sự thỏa thuận cảm tính giữa người mua và người bán. Song điều này dẫn đến sự bất cân xứng thông tin, gây khó khăn cho việc định giá nhanh và chính xác dựa trên quy mô lớn. Một chiếc xe không chỉ mất giá theo thời gian mà còn phụ thuộc vào:

- Thông số kỹ thuật: Loại động cơ, công suất, loại nhiên liệu sử dụng
- Lịch sử vận hành: Số km đã đi, số đời chủ sở hữu

Chính vì vậy, việc xây dựng một mô hình hỗ trợ dự báo giá xe tự động không chỉ giúp tối ưu hóa quy trình tham khảo giá mà còn góp phần tăng tính minh bạch cho thị trường. Đề tài này được thực hiện nhằm nghiên cứu khả năng áp dụng các thuật toán hồi quy (Regression) để xử lý dữ liệu thực tế từ thị trường ô tô cũ, từ đó xây dựng quy trình cơ bản cho bài toán định giá.

1.2. Lý do chọn đề tài

Trong những năm gần đây, thị trường xe ô tô cũ ngày càng phát triển do nhu cầu mua bán, trao đổi phương tiện đã qua sử dụng tăng cao. Tuy nhiên, việc xác định giá trị thực tế của một chiếc xe cũ vẫn còn phụ thuộc nhiều vào kinh nghiệm cá nhân, cảm tính của người mua bán hoặc đánh giá thủ công từ các đơn vị trung gian. Giá xe cũ chịu ảnh hưởng bởi nhiều yếu tố như năm sản xuất, số km đã đi, hãng xe, loại nhiên liệu, hộp số, số chủ sở hữu và đặc biệt là tình trạng ngoại thất của xe.

Trong thực tế, hai chiếc xe có cùng thông số kỹ thuật nhưng tình trạng hư hỏng khác nhau có thể có giá trị chênh lệch đáng kể. Vì vậy, nếu chỉ dự đoán giá dựa trên dữ liệu dạng bảng mà không xét đến tình trạng hư hỏng từ hình ảnh thì kết quả định giá có thể chưa phản ánh đầy đủ giá trị thực của xe.

Xuất phát từ vấn đề đó, em lựa chọn đề tài “Dự đoán giá xe ô tô cũ có xét đến tình trạng hư hỏng ngoại thất bằng Machine Learning và Computer Vision”. Đề tài kết hợp giữa học máy trên dữ liệu bảng và thị giác máy tính trên dữ liệu ảnh, giúp hệ thống không chỉ dự đoán giá cơ bản của xe mà còn phát hiện hư hỏng, đánh giá mức độ nghiêm trọng và đưa ra giá tham khảo sau điều chỉnh.

Đề tài cũng phù hợp với nội dung môn Thực tập cơ sở vì giúp em vận dụng nhiều kiến thức đã học như xử lý dữ liệu, huấn luyện mô hình Machine Learning, sử dụng mô hình học sâu, xây dựng pipeline xử lý và triển khai ứng dụng demo bằng Streamlit.

1.3. Mục tiêu dự án

Mục tiêu chính của dự án là xây dựng một hệ thống demo có khả năng hỗ trợ định giá xe ô tô cũ dựa trên thông tin kỹ thuật của xe và hình ảnh ngoại thất. Hệ thống hướng đến việc tạo ra một quy trình định giá tương đối hoàn chỉnh, bao gồm dự đoán giá cơ bản, phát hiện hư hỏng, phân loại mức độ hư hỏng và đưa ra giá tham khảo sau điều chỉnh.

Các mục tiêu cụ thể của dự án gồm:

- Thu thập, phân tích và tiền xử lý dữ liệu xe ô tô cũ dạng bảng để phục vụ bài toán dự đoán giá.
- Xây dựng và đánh giá các mô hình hồi quy như Linear Regression, Random Forest và XGBoost để dự đoán giá cơ bản của xe.
- Tích hợp mô hình YOLO để phát hiện các vùng hư hỏng trên ảnh xe.
- Sử dụng mô hình CNN/ResNet để phân loại mức độ hư hỏng thành các mức như nhẹ, trung bình và nghiêm trọng.
- Xây dựng cơ chế điều chỉnh giá tham khảo dựa trên số lượng hư hỏng, loại hư hỏng, diện tích vùng hỏng và mức độ nghiêm trọng.
- Phát triển giao diện demo bằng Streamlit, cho phép người dùng nhập thông tin xe, tải ảnh lên và nhận kết quả dự đoán trực quan.
- Đánh giá ưu điểm, hạn chế của hệ thống và đề xuất hướng phát triển trong tương lai.

Thông qua dự án này, em mong muốn hiểu rõ hơn quy trình xây dựng một sản phẩm Machine Learning hoàn chỉnh, từ bước chuẩn bị dữ liệu, huấn luyện mô hình, đánh giá kết quả cho đến tích hợp thành một ứng dụng có thể sử dụng được.

1.4. Phạm vi

Dự án được giới hạn trong phạm vi xây dựng một hệ thống demo hỗ trợ ước lượng giá xe ô tô cũ, kết hợp dữ liệu bảng và dữ liệu ảnh ngoại thất. Hệ thống chưa hướng đến triển khai production hoặc thay thế hoàn toàn quy trình thẩm định xe thực tế.

Phạm vi thực hiện:

- Dự đoán giá cơ bản của xe dựa trên dữ liệu dạng bảng gồm các thuộc tính kỹ thuật và lịch sử sử dụng.
- Phát hiện một số dạng hư hỏng ngoại thất phổ biến trên ảnh xe bằng YOLOv8.
- Phân loại mức độ hư hỏng thành các mức nhẹ, trung bình và nghiêm trọng bằng CNN/ResNet18.
- Tính giá tham khảo sau điều chỉnh bằng cơ chế rule-based dựa trên kết quả phát hiện và phân loại hư hỏng.
- Xây dựng giao diện demo bằng Streamlit để minh họa luồng nhập thông tin xe, tải ảnh và hiển thị kết quả.

Giới hạn của phạm vi:

- Các bộ dữ liệu bảng và ảnh chưa đồng bộ hoàn toàn theo từng chiếc xe.
- Dự án chưa có nhãn ground truth cho giá xe sau khi xét đến từng loại hư hỏng cụ thể.
- Giá tham khảo sau điều chỉnh mang tính minh họa cho ý tưởng kết hợp Computer Vision vào bài toán định giá, chưa đại diện cho giá giao dịch thực tế.

Công nghệ sử dụng:

- Ngôn ngữ lập trình: Python.
- Xử lý dữ liệu và trực quan hóa: Pandas, NumPy, Matplotlib, Seaborn.
- Machine Learning: Scikit-learn, XGBoost.
- Deep Learning và Computer Vision: PyTorch, ResNet18, YOLOv8/Ultralytics, OpenCV/PIL.
- Giao diện demo: Streamlit.

2. Cơ sở lý thuyết

2.1. Bài toán hồi quy trong Machine Learning

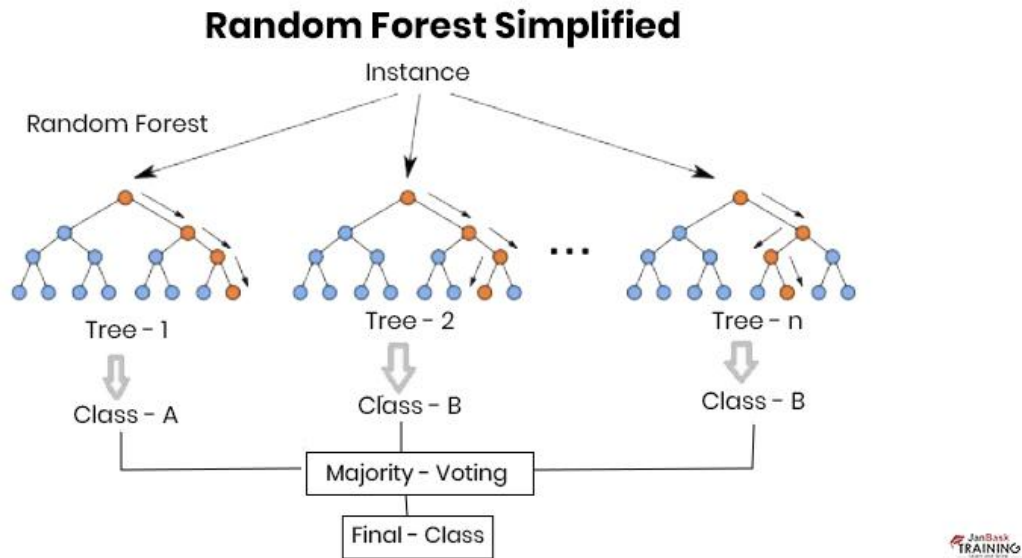
2.2. Random Forest, Linear Regression, XGBoost

2.2.1 Random Forest

2.2.1.1. Khái niệm

Random Forest là một thuật toán học có giám sát, thuộc nhóm các thuật toán ensemble learning. Thuật toán này hoạt động bằng cách xây dựng nhiều cây quyết định(

Decision Tree) trong quá trình đào tạo và đưa ra dự đoán bằng cách tính trung bình các dự đoán của các cây quyết định đó hoặc chọn lớp có số phiếu bầu nhiều nhất trong trường hợp phân loại. Mỗi cây trong Random Forest được xây dựng từ một mẫu có thể của dữ liệu đào tạo(bootstrapping) và trong quá trình xây dựng mỗi cây, chỉ một tập con ngẫu nhiên của các đặc điểm được sử dụng để chia các nút của cây.



2.2.1.2. Cách thức hoạt động

- Bước 1: Chọn điểm dữ liệu K ngẫu nhiên từ tập huấn luyện.
- Bước 2: Xây dựng cây quyết định liên kết với các điểm dữ liệu đã chọn (Tập con).
- Bước 3: Chọn số N cho cây quyết định mà bạn muốn xây dựng.
- Bước 4: Lặp lại Bước 1 và 2.
- Bước 5: Đối với các điểm dữ liệu mới, hãy tìm các dự đoán của từng cây quyết định và gán các điểm dữ liệu mới cho danh mục giành được đa số phiếu bầu.

2.2.1.3. Ưu và nhược điểm của thuật toán

- **Ưu điểm:**
 - **Hiệu quả cao:** Random Forest có khả năng cung cấp kết quả chính xác cao, ngay cả trên các bộ dữ liệu lớn và phức tạp.
 - **Kiểm soát overfitting:** Mặc dù từng cây quyết định có thể bị overfit, kết hợp kết quả từ nhiều cây giúp giảm thiểu vấn đề này.
 - **Dễ dàng đo lường mức độ quan trọng của từng đặc điểm:** Random Forest có thể cung cấp thông tin chi tiết về mức độ quan trọng của các đặc điểm đầu vào đối với quá trình dự đoán.

- **Khả năng xử lý dữ liệu cả bảng và dạng phân loại:** Thuật toán này có thể xử lý cả các biến định lượng và định tính.
- **Nhược điểm:**
 - **Không trực quan và khó giải thích:** Mô hình Random Forest gồm nhiều cây quyết định có thể khó hiểu và giải thích so với một cây quyết định đơn lẻ.
 - **Thời gian đào tạo có thể lâu hơn:** Xây dựng nhiều cây đòi hỏi nhiều tài nguyên tính toán hơn, đặc biệt là trên các bộ dữ liệu lớn.
 - **Độ phức tạp cao khi triển khai:** Khi số lượng cây trong rừng tăng lên, mô hình cần nhiều bộ nhớ hơn và thời gian phản hồi có thể chậm hơn.

2.2.1.4. Vai trò của Random Forest trong dự án

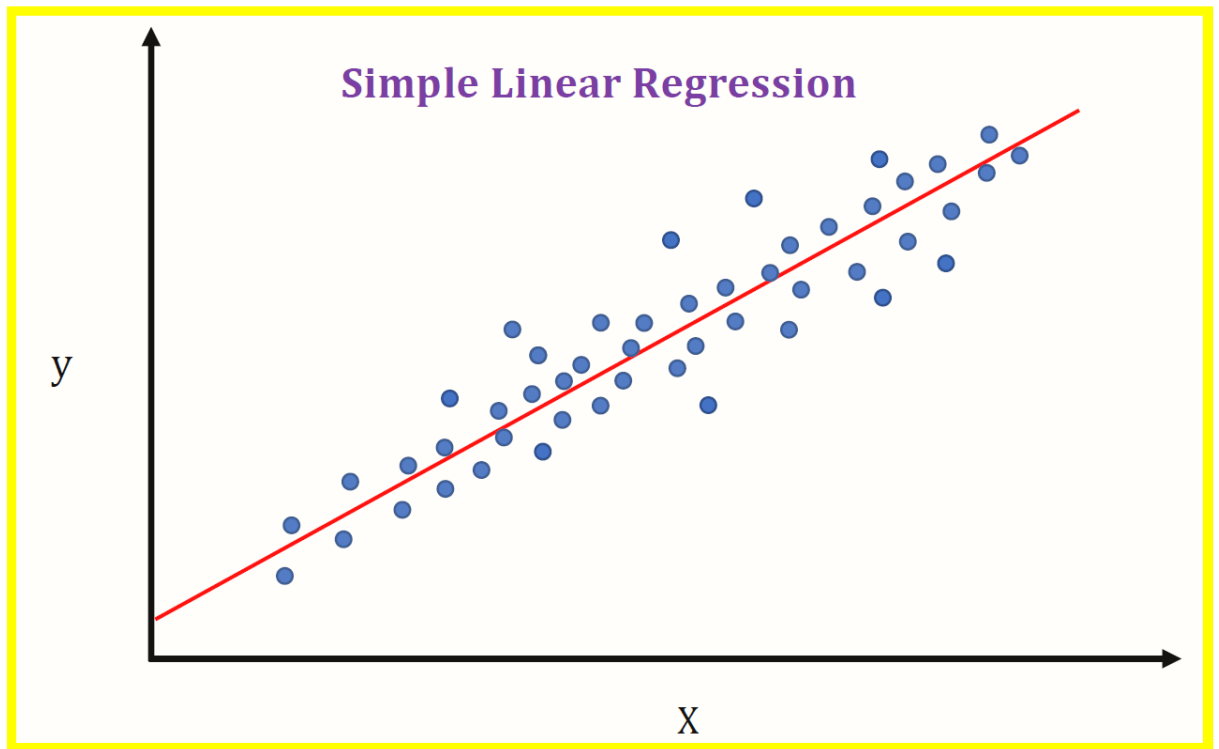
Trong dự án, Random Forest được sử dụng như một mô hình hồi quy phi tuyến để so sánh với Linear Regression và XGBoost trên dữ liệu giá xe cũ. Thuật toán này phù hợp với dữ liệu bảng vì có thể học các quan hệ phức tạp giữa tuổi xe, số km đã đi, công suất, dung tích động cơ, loại nhiên liệu và giá bán.

Ngoài vai trò dự đoán, Random Forest còn giúp tham khảo mức độ quan trọng của các đặc trưng đầu vào. Tuy nhiên, do mô hình gồm nhiều cây quyết định nên khả năng giải thích chi tiết từng dự đoán không trực quan bằng mô hình tuyến tính.

2.2.2 Linear Regression

2.2.2.1. Khái niệm

Hồi quy tuyến tính (Linear Regression) là một thuật toán học máy có giám sát được sử dụng để dự đoán giá trị của một biến mục tiêu (biến phụ thuộc) dựa trên một hoặc nhiều biến dự báo (biến độc lập). Thuật toán này dựa trên giả định rằng có một mối quan hệ tuyến tính (đường thẳng) giữa các biến đầu vào và kết quả đầu ra. Kết quả của mô hình là một giá trị liên tục (định lượng), ví dụ như giá cả, điểm số hoặc nhiệt độ.



2.2.2.2. Cách thức hoạt động

- **Tìm đường tiệm cận tốt nhất:** Thuật toán cố gắng vẽ một đường thẳng (hoặc siêu phẳng trong không gian nhiều chiều) đi qua các điểm dữ liệu sao cho tổng sai số giữa giá trị dự đoán và giá trị thực tế là nhỏ nhất.
- **Phương pháp Bình phương tối thiểu (OLS):** Đây là kỹ thuật phổ biến nhất để tìm ra các hệ số của phương pháp hồi quy bằng cách tối thiểu hóa tổng bình phương của các khoảng cách sai lệch (phần dư).
- **Hệ số hồi quy:** Mỗi biến đầu vào sẽ được gán một trọng số, thể hiện mức độ và chiều hướng ảnh hưởng của biến đó đến kết quả cuối cùng.

2.2.2.3. Ưu và nhược điểm của thuật toán

- **Ưu điểm:**
 - **Đơn giản và dễ giải thích:** Kết quả của mô hình rất trực quan, giúp người dùng hiểu rõ biến nào quan trọng và ảnh hưởng như thế nào đến kết quả.
 - **Hiệu suất tính toán cao:** Thuật toán chạy rất nhanh, tốn ít tài nguyên và dễ dàng triển khai trên các hệ thống thực tế.
 - **Hiệu quả với dữ liệu tuyến tính:** Nếu mối quan hệ giữa các biến là tuyến tính, mô hình này thường cho kết quả rất ổn định.

- **Nhược điểm:**

- **Giới hạn về tính tuyến tính:** Thuật toán không thể học được các mối quan hệ phức tạp hoặc phi tuyến giữa các dữ liệu.
- **Nhạy cảm với ngoại lệ (Outliers):** Chỉ cần một vài điểm dữ liệu có giá trị bất thường cũng có thể làm lệch hướng đường hồi quy, dẫn đến sai số lớn.
- **Hiện tượng đa cộng tuyến:** Nếu các biến độc lập có liên quan quá chặt chẽ với nhau, mô hình có thể trở nên thiếu chính xác

2.2.2.4. Vai trò của Linear Regression trong dự án

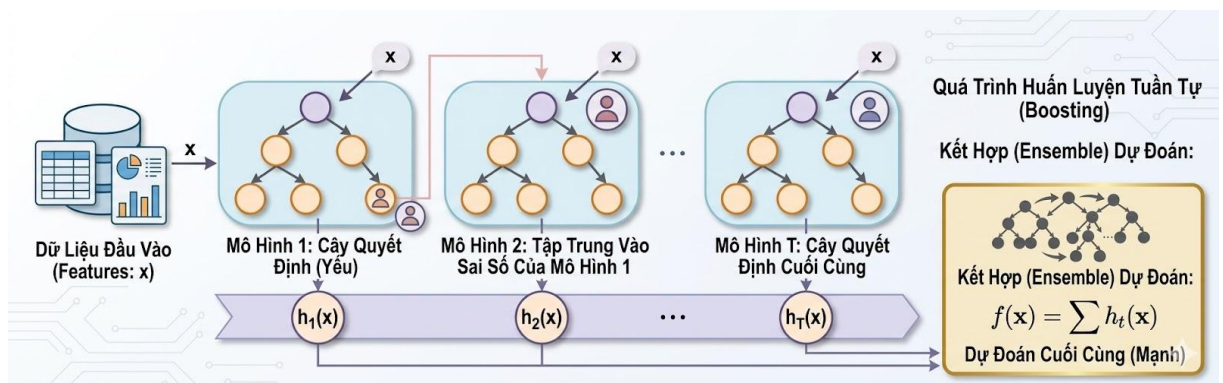
Trong dự án, Linear Regression được dùng làm mô hình baseline cho bài toán dự đoán giá xe. Mục đích chính của mô hình này là tạo ra một mốc so sánh đơn giản để đánh giá xem các mô hình phi tuyến như Random Forest và XGBoost cải thiện kết quả đến mức nào.

Do bài toán giá xe cũ chịu ảnh hưởng bởi nhiều yếu tố phi tuyến như tuổi xe, số km đã đi, hãng xe và loại nhiên liệu, Linear Regression thường khó đạt kết quả tốt nhất. Tuy vậy, mô hình này vẫn có giá trị vì dễ huấn luyện, dễ giải thích và giúp kiểm tra nhanh xu hướng của dữ liệu.

2.2.3. XGBoost

2.2.3.1. Khái niệm

XGBoost (Extreme Gradient Boosting) là một thuật toán học máy mạnh mẽ được phát triển dựa trên thuật toán Gradient Boosting Trees (Cây Boosting Gradient). XGBoost được biết đến với khả năng học tập hiệu quả, chính xác cao và có thể giải quyết được nhiều loại bài toán học máy khác nhau, đặc biệt là các bài toán phân loại và hồi quy.



2.2.3.2. Cách thức hoạt động

- **Xây dựng tuần tự (Sequential Building):** Khác với Random Forest xây dựng các cây độc lập, XGBoost xây dựng các cây quyết định

một cách tuần tự. Mỗi cây mới được thêm vào sẽ cố gắng học và sửa chữa những sai số (residuals) mà các cây trước đó đã mắc phải.

- **Tối ưu hóa Gradient:** Thuật toán sử dụng phương pháp hạ độ dốc (Gradient Descent) để giảm thiểu hàm mất mát. Điểm đặc biệt của XGBoost là sử dụng khai triển Taylor bậc hai để xấp xỉ hàm mất mát, giúp quá trình tìm ra cấu trúc cây tối ưu diễn ra nhanh và chính xác hơn.
- **Cơ chế cắt tỉa (Pruning):** XGBoost sử dụng tham số "gain" để quyết định xem một nút có nên được chia hay không. Nó xây dựng cây đến độ sâu tối đa trước, sau đó mới cắt tỉa ngược lại từ dưới lên để loại bỏ những nhánh không mang lại giá trị đáng kể cho mô hình.
- **Xử lý dữ liệu khuyết (Sparsity-aware splitting):** Thuật toán có khả năng tự động học hướng phân tách tốt nhất khi gặp các giá trị bị thiếu trong dữ liệu, giúp mô hình hoạt động ổn định mà không cần xử lý dữ liệu trống quá phức tạp

2.2.3.3. Ưu và nhược điểm của thuật toán

- Ưu điểm:

- **Hiệu suất cao và nhanh chóng:** XGBoost được tối ưu hóa để hiệu quả cả về tốc độ lẫn sử dụng bộ nhớ, thường nhanh hơn các thuật toán gradient boosting truyền thống.
- **Khả năng xử lý quy mô lớn:** Thuật toán có thể xử lý hàng triệu dữ liệu và tương thích tốt với các nền tảng tính toán song song và phân tán.
- **Chống quá khớp:** XGBoost cung cấp các tính năng như cắt tỉa cây (pruning), điều chỉnh độ sâu của cây, và định thời (regularization), giúp tránh hiện tượng quá khớp một cách hiệu quả.
- **Tính linh hoạt:** Có thể tùy chỉnh được nhiều tham số và hỗ trợ các hàm mục tiêu tùy chỉnh và hàm đánh giá trong quá trình huấn luyện.
- **Xử lý các tính năng dạng số và dạng danh mục một cách tự động.**

- Nhược điểm:

- **Đòi hỏi kiến thức kỹ thuật:** Việc căn phải hiệu và tinh chỉnh nhiều tham số có thể làm cho XGBoost khó sử dụng hiệu quả đối với người mới bắt đầu.

- **Thời gian huấn luyện có thể lâu:** Dù được tối ưu hóa, nhưng việc xây dựng hàng trăm cây có thể tốn thời gian, đặc biệt là trên các bộ dữ liệu cực lớn.
- **Không phù hợp cho dữ liệu có chiều dữ liệu cao và thưa thớt:** Trong một số trường hợp, các mô hình tuyến tính có thể hiệu quả hơn đối với dữ liệu thưa.

2.2.3.4. Vai trò của XGBoost trong dự án

Trong dự án, XGBoost được lựa chọn là mô hình chính cho nhánh dự đoán giá cơ bản vì có khả năng xử lý tốt dữ liệu bảng, học được quan hệ phi tuyến và thường đạt hiệu quả cao trong các bài toán regression.

So với Linear Regression, XGBoost biểu diễn tốt hơn các tương tác phức tạp giữa các đặc trưng. So với Random Forest, XGBoost có cơ chế học tuần tự để sửa sai qua từng cây, nhờ đó đạt kết quả tốt hơn trong phần thực nghiệm của dự án.

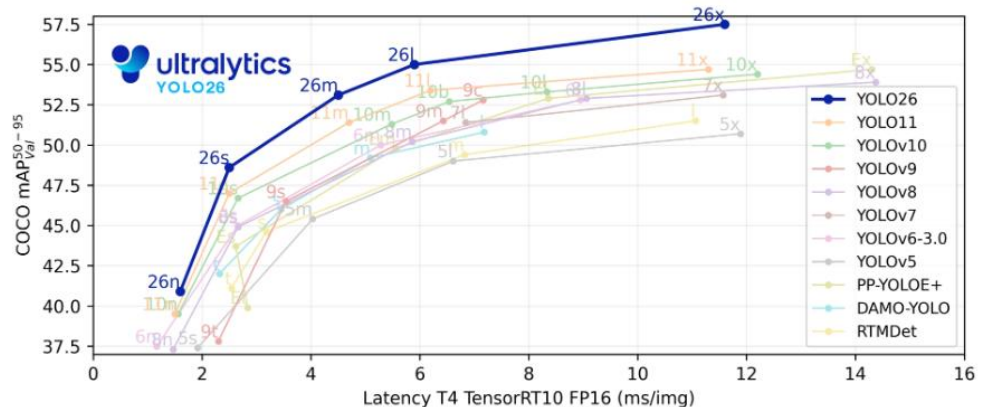
2.3. YOLO cho object detection

Trong dự án, mô hình YOLO được sử dụng cho bài toán phát hiện đối tượng, cụ thể là phát hiện các vùng hư hỏng trên ảnh xe ô tô. YOLO, viết tắt của “You Only Look Once”, là một mô hình phát hiện đối tượng thuộc nhóm one-stage detector. Khác với các phương pháp phát hiện đối tượng truyền thống cần nhiều bước xử lý, YOLO có khả năng xác định vị trí và phân loại đối tượng trong cùng một lần suy luận. Nhờ đó, mô hình có tốc độ xử lý nhanh và phù hợp với các ứng dụng cần phản hồi gần thời gian thực.

Ultralytics YOLO26

Overview

Ultralytics YOLO26 is the latest evolution in the YOLO series of real-time object detectors, engineered from the ground up for **edge and low-power devices**. It introduces a streamlined design that removes unnecessary complexity while integrating targeted innovations to deliver faster, lighter, and more accessible deployment.



[Try on Ultralytics Platform](#)

Explore and run YOLO26 models directly on [Ultralytics Platform](#).

Lý do lựa chọn YOLO trong dự án là vì mô hình này có sự cân bằng tốt giữa tốc độ và độ chính xác. Đối với bài toán đánh giá tình trạng xe cũ, hệ thống cần phát hiện được các hư hỏng ngoại thất như vết móp, vết xước, nứt vỡ hoặc hư hỏng đèn kính từ ảnh do người dùng cung cấp. YOLO cho phép phát hiện trực tiếp các vùng hư hỏng này thông qua bounding box, đồng thời trả về nhãn loại hư hỏng và độ tin cậy của từng dự đoán.

Trong hệ thống, YOLO đóng vai trò là mô-đun thị giác máy tính đầu tiên trong pipeline xử lý ảnh. Sau khi người dùng tải ảnh xe lên giao diện, ảnh sẽ được đưa vào mô hình YOLO để phát hiện các vùng hư hỏng. Kết quả đầu ra của mô hình bao gồm tọa độ bounding box, loại hư hỏng, confidence score và tỷ lệ diện tích vùng hư hỏng so với toàn bộ ảnh. Các thông tin này không chỉ được dùng để hiển thị trực quan trên giao diện mà còn là đầu vào cho các bước xử lý tiếp theo.

Các lớp hư hỏng được sử dụng trong dự án bao gồm một số loại phổ biến như dent, scratch, crack, lamp_broken, glass_broken và tire_flat. Dữ liệu huấn luyện được tổ chức theo định dạng YOLO, trong đó mỗi ảnh đi kèm với file nhãn chứa thông tin về class và tọa độ vùng hư hỏng. Mô hình sau khi huấn luyện được lưu lại dưới dạng file trọng số và được tích hợp vào hệ thống để phục vụ quá trình suy luận.

Sau khi YOLO phát hiện các vùng hư hỏng, hệ thống sẽ vẽ bounding box lên ảnh để người dùng có thể quan sát trực tiếp kết quả. Đồng thời, từng vùng hư hỏng được đưa sang mô hình phân loại mức độ nghiêm trọng nhằm xác định hư hỏng thuộc mức nhẹ, trung bình hay nghiêm trọng. Cuối cùng, thông tin về loại hư hỏng, số lượng hư hỏng, diện tích vùng hỏng và mức độ nghiêm trọng được sử dụng trong bước điều chỉnh giá xe.

Nhờ việc tích hợp YOLO, hệ thống không chỉ dự đoán giá xe dựa trên dữ liệu dạng bảng mà còn có khả năng khai thác thông tin từ hình ảnh thực tế của xe. Điều này giúp kết quả định giá phản ánh tốt hơn tình trạng ngoại thất, từ đó làm cho hệ thống gần với bài toán định giá xe cũ trong thực tế hơn.

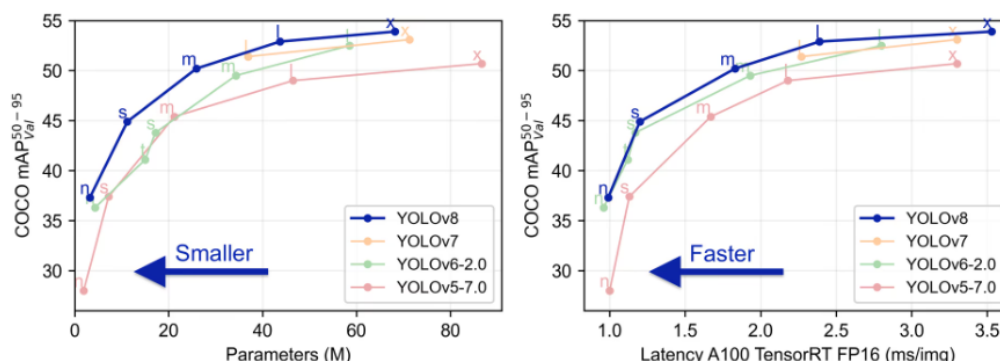
YOLOv8 hỗ trợ tốt cho bài toán phát hiện nhiều loại hư hỏng trên ảnh xe trong cùng một khung hình. Trong dự án, hệ thống cần nhận diện các vùng hư hỏng ngoại thất. Đây là các dạng hư hỏng có đặc điểm thị giác khác nhau: vết xước thường nhỏ và mảnh, vết móp có thể biến dạng theo bề mặt thân xe, trong khi kính hoặc đèn vỡ thường có hình dạng rõ hơn. YOLOv8 cho phép học đồng thời nhiều lớp hư hỏng như vậy, đồng thời trả về vị trí bounding box, nhãn hư hỏng và độ tin cậy của từng dự đoán. Điều này phù hợp với yêu cầu của hệ thống vì kết quả phát hiện không chỉ dùng để hiển thị trực quan mà còn được đưa sang các bước phân loại mức độ hư hỏng và điều chỉnh giá xe.



Explore Ultralytics YOLOv8

Overview

YOLOv8 was released by Ultralytics on January 10, 2023, offering cutting-edge performance in terms of accuracy and speed. Building upon the advancements of previous YOLO versions, YOLOv8 introduced new features and optimizations that make it an ideal choice for various [object detection](#) tasks in a wide range of applications.



YOLOv8 có nhiều biến thể kích thước khác nhau như YOLOv8n, YOLOv8s và YOLOv8m, cho phép thử nghiệm và đánh đổi giữa tốc độ với độ

chính xác. Trong quá trình huấn luyện, dự án đã so sánh nhiều biến thể YOLOv8 trên bộ dữ liệu phát hiện hư hỏng.

- YOLOv8n có ưu điểm nhẹ và suy luận nhanh nhưng độ chính xác tổng thể chưa phải tốt nhất.
- YOLOv8m có khả năng học mạnh hơn nhưng kích thước mô hình và chi phí tính toán cao hơn, không thật sự phù hợp với mục tiêu tích hợp vào ứng dụng demo.
- YOLOv8s cho thấy mức cân bằng tốt hơn giữa độ chính xác, tốc độ xử lý và khả năng triển khai, vì vậy checkpoint yolov8_s100_640 được chọn làm mô hình phát hiện hư hỏng chính của hệ thống. Việc lựa chọn YOLOv8 phù hợp với định hướng nghiên cứu và triển khai của dự án.

Performance

Detection (COCO) Detection (Open Images V7) Segmentation (COCO) Classification (ImageNet) Pose (COCO) OBB (DC)

See [Detection Docs](#) for usage examples with these models trained on [COCO](#), which include 80 pretrained classes.

Model	size (pixels)	mAP ^{val} 50-95	Speed CPU ONNX (ms)	Speed A100 TensorRT (ms)	params (M)	FLOPs (B)
YOLOv8n	640	37.3	80.4	0.99	3.2	8.7
YOLOv8s	640	44.9	128.4	1.20	11.2	28.6
YOLOv8m	640	50.2	234.7	1.83	25.9	78.9
YOLOv8l	640	52.9	375.2	2.39	43.7	165.2
YOLOv8x	640	53.9	479.1	3.53	68.2	257.8

Thông qua mô hình này, dự án có thể tiếp cận đầy đủ quy trình của một bài toán Computer Vision hiện đại, từ chuẩn bị dữ liệu ảnh, chuyển đổi annotation sang định dạng YOLO, huấn luyện mô hình, đánh giá bằng các chỉ số như **Precision**, **Recall**, **mAP**, đến tích hợp mô hình vào pipeline hoàn chỉnh.

Việc tìm hiểu về YOLOv8 coi như là bước đầu cho việc tìm hiểu sâu hơn về Computer Vision.

2.4. CNN/ResNet18 cho phân loại ảnh

CNN được sử dụng trong dự án để phân loại mức độ nghiêm trọng của hư hỏng sau khi YOLO đã phát hiện vùng hỏng trên ảnh xe. Nếu YOLO đóng vai trò xác định “hư hỏng nằm ở đâu” và “thuộc loại nào”, thì CNN tiếp tục xử lý vùng ảnh đó để đánh giá “mức độ hư hỏng nghiêm trọng ra sao”. Trong dự án, các vùng hư hỏng được phân loại thành ba mức chính là **minor, moderate và severe**, tương ứng với hư hỏng nhẹ, trung bình và nghiêm trọng. Kết quả này giúp hệ thống có thêm cơ sở để điều chỉnh giá xe hợp lý hơn thay vì chỉ dựa vào số lượng hư hỏng.

Dự án lựa chọn ResNet18 làm kiến trúc CNN chính vì đây là mô hình gọn, ổn định và phù hợp với quy mô bài toán. ResNet18 sử dụng cơ chế residual connection, giúp mô hình học đặc trưng ảnh hiệu quả hơn và hạn chế hiện tượng suy giảm hiệu năng khi mạng sâu hơn. So với các kiến trúc lớn như ResNet50 hoặc EfficientNet phiên bản lớn, ResNet18 có số tham số ít hơn, tốc độ suy luận nhanh hơn và dễ tích hợp vào pipeline demo. Điều này phù hợp với mục tiêu của dự án là xây dựng một hệ thống thực nghiệm có thể chạy được trên máy cá nhân hoặc môi trường triển khai đơn giản.

Trong pipeline của hệ thống, ResNet18 nhận đầu vào là ảnh crop từ vùng bounding box do YOLO phát hiện. Các crop này được chuẩn hóa kích thước và tiền xử lý trước khi đưa vào mô hình phân loại. Việc sử dụng crop vùng hư hỏng thay vì toàn bộ ảnh giúp mô hình tập trung hơn vào đặc trưng cần phân tích, giảm nhiễu từ nền ảnh hoặc các bộ phận không liên quan của xe. Sau khi phân loại, mỗi vùng hư hỏng sẽ được gán một nhãn mức độ và một điểm severity score để phục vụ bước tổng hợp kết quả.

Kết quả phân loại mức độ hư hỏng được sử dụng trực tiếp trong tầng điều chỉnh giá. Cùng một loại hư hỏng nhưng nếu mức độ khác nhau thì ảnh hưởng đến giá xe cũng khác nhau. Ví dụ, một vết xước nhỏ chỉ nên làm giảm giá nhẹ, trong khi một vết nứt hoặc móp nặng có thể khiến mức giảm giá cao hơn. Vì vậy, việc bổ sung mô hình CNN/ResNet18 giúp hệ thống đánh giá tình trạng ngoại thất chi tiết hơn, làm cho kết quả định giá tham khảo cuối cùng có tính giải thích tốt hơn.

Việc kết hợp YOLOv8 và ResNet18 tạo thành một nhánh xử lý ảnh tương đối hoàn chỉnh trong dự án. YOLOv8 đảm nhiệm nhiệm vụ phát hiện vị trí và loại hư hỏng, còn ResNet18 đảm nhiệm nhiệm vụ đánh giá mức độ nghiêm trọng của từng vùng hư hỏng. Cách chia nhỏ bài toán thành hai bước giúp hệ thống dễ huấn luyện, dễ kiểm tra lỗi và dễ thay thế từng mô hình trong tương lai.

Đây cũng là cách tiếp cận phù hợp với một dự án thực nghiệm, khi dữ liệu giá cuối sau hư hỏng chưa đồng bộ hoàn toàn với dữ liệu ảnh.

2.5. Các metric: RMSE, MAE, R^2 , mAP, accuracy

Trong dự án, hệ thống gồm nhiều mô hình đảm nhiệm các nhiệm vụ khác nhau, bao gồm mô hình hồi quy để dự đoán giá xe, mô hình phát hiện hư hỏng bằng YOLO và mô hình phân loại mức độ hư hỏng bằng CNN/ResNet18. Vì mỗi bài toán có bản chất khác nhau nên cần sử dụng các nhóm chỉ số đánh giá khác nhau để phản ánh đúng hiệu quả của từng thành phần.

2.5.1. Metrics Cho Bài Toán Dự Đoán Giá Xe

Đối với mô hình dự đoán giá xe từ dữ liệu dạng bảng, đây là bài toán hồi quy nên dự án sử dụng các chỉ số như MAE, RMSE và R^2 Score.

- MAE, viết tắt của Mean Absolute Error, là sai số tuyệt đối trung bình giữa giá trị dự đoán và giá trị thực tế. Chỉ số này cho biết trung bình mô hình dự đoán lệch bao nhiêu so với giá trị thật. MAE càng nhỏ thì mô hình càng dự đoán gần với thực tế. Trong bài toán định giá xe cũ, MAE giúp đánh giá mức sai lệch trung bình của mô hình theo đơn vị giá.
- RMSE, viết tắt của Root Mean Squared Error, là căn bậc hai của trung bình bình phương sai số. So với MAE, RMSE nhạy hơn với các dự đoán sai lệch lớn vì sai số được bình phương trước khi lấy trung bình. Do đó, RMSE phù hợp để kiểm tra xem mô hình có tạo ra những dự đoán lệch quá xa so với giá trị thật hay không. RMSE càng nhỏ thì mô hình càng ổn định.
- R^2 Score là hệ số xác định, dùng để đo mức độ mô hình giải thích được sự biến thiên của dữ liệu mục tiêu. Giá trị R^2 càng gần 1 thì mô hình càng phù hợp với dữ liệu. Trong dự án, R^2 được dùng để so sánh khả năng học quan hệ giữa các đặc trưng như năm sản xuất, số km đã đi, loại nhiên liệu, hộp số, hãng xe và giá xe.

2.5.2 Metrics Cho Bài Toán Object Detection Bằng YOLO

Đối với mô hình YOLOv8, nhiệm vụ của mô hình không chỉ là phân loại đúng loại hư hỏng mà còn phải xác định đúng vị trí vùng hư hỏng trên ảnh. Vì vậy, các chỉ số đánh giá thường dùng gồm Precision, Recall, mAP@0.5 và [mAP@0.5:0.95](#).

Precision cho biết trong số các vùng hư hỏng mà mô hình dự đoán, có bao nhiêu vùng là đúng. Precision cao nghĩa là mô hình ít báo nhầm. Trong bài toán phát hiện hư hỏng xe, điều này giúp giảm tình trạng nhận nhầm các chi tiết bình thường như bóng phản sáng, đường viền thân xe hoặc vết bẩn thành hư hỏng.

Recall cho biết trong số các vùng hư hỏng thực sự tồn tại trên ảnh, mô hình phát hiện được bao nhiêu vùng. Recall cao nghĩa là mô hình ít bỏ sót hư hỏng. Đây là chỉ số quan trọng vì nếu hệ thống bỏ sót nhiều vùng hỏng thì bước điều chỉnh giá sau đó sẽ thiếu thông tin.

$mAP@0.5$, hay mean Average Precision tại ngưỡng IoU 0.5, là chỉ số tổng hợp giữa khả năng phân loại đúng và định vị đúng bounding box. Một dự đoán được xem là đúng khi bounding box dự đoán có độ giao nhau đủ lớn với bounding box thật. Trong dự án, $mAP@0.5$ được dùng làm chỉ số chính để đánh giá chất lượng phát hiện hư hỏng của YOLO.

$mAP@0.5:0.95$ là phiên bản đánh giá khắt khe hơn, tính trung bình mAP trên nhiều ngưỡng IoU từ 0.5 đến 0.95. Chỉ số này phản ánh tốt hơn độ chính xác của vị trí bounding box. Nếu $mAP@0.5:0.95$ cao, điều đó cho thấy mô hình không chỉ phát hiện đúng vùng hỏng mà còn khoanh vùng tương đối chính xác.

2.5.3. Metrics cho bài toán phân loại mức độ hư hỏng

Đối với mô hình CNN/ResNet18, nhiệm vụ là phân loại ảnh vùng hư hỏng thành các mức minor, moderate và severe. Đây là bài toán phân loại nhiều lớp, vì vậy có thể sử dụng các chỉ số như Accuracy, Precision, Recall, F1-score và Confusion Matrix.

Accuracy là tỷ lệ dự đoán đúng trên tổng số mẫu. Đây là chỉ số dễ hiểu và phù hợp để đánh giá tổng quan mô hình phân loại. Tuy nhiên, nếu dữ liệu giữa các lớp không cân bằng, Accuracy có thể chưa phản ánh đầy đủ chất lượng mô hình.

Precision trong bài toán phân loại cho biết trong số các mẫu được dự đoán thuộc một mức độ hư hỏng, có bao nhiêu mẫu thực sự thuộc mức đó. Chỉ số này giúp đánh giá mức độ đáng tin cậy của từng nhãn mà mô hình đưa ra.

Recall cho biết trong số các mẫu thực sự thuộc một mức độ hư hỏng, mô hình nhận diện đúng được bao nhiêu mẫu. Với dự án này, Recall đặc biệt quan trọng ở lớp severe, vì bỏ sót các hư hỏng nghiêm trọng có thể khiến hệ thống đánh giá mức giảm giá thấp hơn thực tế.

F1-score là trung bình điều hòa giữa Precision và Recall. Chỉ số này hữu ích khi cần đánh giá cân bằng giữa việc hạn chế dự đoán sai và hạn chế bỏ sót. Với bài toán phân loại mức độ hư hỏng, F1-score giúp đánh giá mô hình tốt hơn so với chỉ dùng Accuracy.

Confusion Matrix là ma trận thể hiện số lượng mẫu của từng lớp được dự đoán đúng hoặc nhầm sang lớp khác. Ma trận này giúp quan sát cụ thể mô hình

thường nằm giữa những mức nào, ví dụ nằm minor với moderate hoặc moderate với severe. Đây là thông tin quan trọng để cải thiện dữ liệu và mô hình trong các phiên bản sau.

2.5.4. Vai trò của metrics trong dự án

Việc sử dụng nhiều nhóm metrics giúp đánh giá hệ thống một cách toàn diện hơn. Mô hình dự đoán giá được đánh giá bằng các chỉ số hồi quy, YOLO được đánh giá bằng các chỉ số phát hiện đối tượng, còn CNN được đánh giá bằng các chỉ số phân loại ảnh. Nhờ đó, dự án có thể xác định rõ thành phần nào hoạt động tốt, thành phần nào còn hạn chế và cần cải thiện.

Trong hệ thống hiện tại, kết quả cuối cùng phụ thuộc vào cả ba nhánh: dự đoán giá cơ bản, phát hiện hư hỏng và phân loại mức độ hư hỏng. Nếu một trong các mô hình hoạt động chưa tốt, kết quả định giá tham khảo cuối cùng cũng có thể bị ảnh hưởng. Vì vậy, việc theo dõi các metrics riêng cho từng mô hình là cần thiết để đảm bảo pipeline hoạt động ổn định và có cơ sở đánh giá rõ ràng.

3. Dataset và tiền xử lý

Trong dự án, dữ liệu được tổ chức theo hướng mô-đun, tương ứng với các nhiệm vụ khác nhau trong pipeline định giá xe ô tô cũ. Thay vì sử dụng một bộ dữ liệu duy nhất cho toàn bộ hệ thống, dự án kết hợp ba nhóm dữ liệu chính: dữ liệu bảng phục vụ dự đoán giá xe cơ bản, dữ liệu ảnh phục vụ phát hiện hư hỏng ngoại thất bằng YOLOv8, và dữ liệu ảnh phục vụ phân loại mức độ hư hỏng bằng CNN/ResNet18.

Cách tổ chức này phù hợp với mục tiêu của dự án vì bài toán định giá xe cũ có xét đến hư hỏng là một bài toán gồm nhiều thành phần. Dữ liệu bảng giúp mô hình học mối quan hệ giữa các thông tin kỹ thuật của xe và giá thị trường. Trong khi đó, dữ liệu ảnh giúp hệ thống khai thác thêm tình trạng ngoại thất thực tế của xe, bao gồm vị trí hư hỏng, loại hư hỏng và mức độ nghiêm trọng.

3.1. Dataset giá xe cũ dạng bảng

Dataset giá xe cũ được lưu dưới dạng file CSV, bao gồm các thông tin mô tả xe như tên xe, năm sản xuất, số km đã đi, loại nhiên liệu, hộp số, số chủ sở hữu, số chỗ ngồi, mức tiêu hao nhiên liệu, dung tích động cơ, công suất và giá xe. Trong đó, cột giá xe được sử dụng làm biến mục tiêu cho bài toán hồi quy.

Mục đích của dataset này là huấn luyện mô hình dự đoán base price, tức là mức giá cơ bản của xe trước khi xét đến tình trạng hư hỏng ngoại thất. Đây là đầu vào quan trọng cho bước tính giá tham khảo sau điều chỉnh của hệ thống.

Trong quá trình tiền xử lý, dữ liệu bảng được làm sạch qua nhiều bước. Trước tiên, các cột được đổi tên về dạng thống nhất để thuận tiện cho quá trình xử lý. Những cột chứa giá trị kèm đơn vị như Mileage, Engine, Power được tách phần số và chuyển sang kiểu dữ liệu số. Các dòng dữ liệu không hợp lệ hoặc nằm ngoài phạm vi hợp lý, chẳng hạn năm sản xuất quá cũ hoặc số chỗ ngồi bất thường, được loại bỏ.

Sau đó, dự án xử lý các giá trị thiếu bằng các phương pháp phù hợp với từng nhóm thuộc tính. Với các cột số, giá trị thiếu được thay thế bằng trung vị để hạn chế ảnh hưởng của outlier. Với các cột phân loại, giá trị thiếu được xử lý bằng giá trị phổ biến nhất hoặc nhóm Unknown tùy từng trường hợp. Các biến phân loại như loại nhiên liệu, hộp số và số đời chủ sở hữu được mã hóa để có thể đưa vào mô hình Machine Learning.

Bên cạnh các thuộc tính gốc, dự án cũng tạo thêm một số đặc trưng mới nhằm cải thiện khả năng học của mô hình. Ví dụ, từ năm sản xuất có thể tính ra Tuổi_xe, từ số km đã đi và tuổi xe có thể tính ra Km_moi_nam. Ngoài ra, dự án còn tạo biến Chay_nhieu để biểu diễn xe có số km đã đi cao hơn mức trung vị hay không, và log_Quang_duong_da_di(km) để giảm độ lệch của phân phối số km. Tên xe cũng được tách ra để lấy thông tin hãng xe và nhóm các dòng xe ít xuất hiện vào nhóm Other.

Cuối cùng, các outlier ở những cột số quan trọng như giá xe, số km đã đi, dung tích động cơ, công suất và mức tiêu hao nhiên liệu được xử lý bằng phương pháp giới hạn theo ngưỡng IQR. Sau bước tiền xử lý, dữ liệu được đưa vào pipeline huấn luyện các mô hình hồi quy như Linear Regression, Random Forest và XGBoost.

3.2. Dataset phát hiện hư hỏng xe

Dataset phát hiện hư hỏng được sử dụng để huấn luyện mô hình YOLOv8 cho bài toán object detection. Dữ liệu bao gồm ảnh xe và annotation mô tả vị trí các vùng hư hỏng trên ảnh. Mỗi vùng hư hỏng được biểu diễn bằng bounding box và nhãn class tương ứng.

Các lớp hư hỏng trong dataset bao gồm những dạng hư hỏng thường gặp trên xe như vết móp, vết xước, vết nứt, kính vỡ hoặc đèn hỏng. Đây là các loại

hư hỏng có đặc điểm thị giác khác nhau, trong đó một số lớp như scratch, crack, dent thường khó phát hiện hơn vì vùng hỏng nhỏ, mảnh hoặc dễ bị nhầm với phản sáng và đường nét thân xe.

Dữ liệu ban đầu được chuyển đổi sang định dạng YOLO để tương thích với quá trình huấn luyện bằng thư viện Ultralytics. Trong định dạng này, mỗi ảnh sẽ có một file nhãn tương ứng, chứa thông tin class và tọa độ bounding box đã được chuẩn hóa. Dataset được chia thành các tập train, val và test, trong đó tập train dùng để học tham số, tập validation dùng để theo dõi chất lượng mô hình trong quá trình huấn luyện, còn tập test dùng để đánh giá cuối cùng.

Trong quá trình tiền xử lý, ảnh và nhãn được kiểm tra để đảm bảo annotation khớp với ảnh, class hợp lệ và không thiếu file nhãn quan trọng. Ảnh được resize theo kích thước đầu vào của mô hình, cụ thể trong dự án là cấu hình yolov8_merge_data_1061, sử dụng YOLOv8s, huấn luyện 100 epoch với kích thước ảnh 640px. Kích thước này giúp giữ lại nhiều chi tiết hơn cho các vùng hư hỏng nhỏ.

3.3. Dataset phân loại mức độ hư hỏng

Bên cạnh việc phát hiện vị trí và loại hư hỏng, dự án còn sử dụng một dataset ảnh khác để huấn luyện mô hình CNN/ResNet18 phân loại mức độ nghiêm trọng của hư hỏng. Dataset này được tổ chức theo các nhãn mức độ như minor, moderate và severe, tương ứng với hư hỏng nhẹ, trung bình và nghiêm trọng.

Mục đích của dataset này là giúp hệ thống không chỉ biết xe bị hư hỏng ở đâu, mà còn đánh giá được mức độ ảnh hưởng của từng vùng hư hỏng. Điều này rất quan trọng trong bài toán định giá, vì cùng một loại hư hỏng nhưng mức độ khác nhau sẽ dẫn đến mức giảm giá khác nhau.

Trong quá trình tiền xử lý, ảnh được resize về kích thước phù hợp với đầu vào của ResNet18 và được normalize theo chuẩn của mô hình pretrained. Nếu áp dụng data augmentation, các phép biến đổi như xoay nhẹ, lật ảnh, thay đổi độ sáng hoặc crop có thể được sử dụng để tăng khả năng tổng quát hóa của mô hình. Khi tích hợp vào pipeline thực tế, các vùng hư hỏng do YOLO phát hiện sẽ được crop theo bounding box, sau đó đưa vào mô hình CNN để dự đoán mức độ hư hỏng.

3.4. Nhận xét về dữ liệu

Một điểm cần lưu ý là ba nhóm dữ liệu trong dự án không đồng bộ hoàn toàn theo từng chiếc xe. Dataset giá xe dùng để học quan hệ giữa thông tin kỹ thuật và giá thị trường, trong khi dataset ảnh hư hỏng dùng để học đặc trưng thị giác liên quan đến tình trạng ngoại thất. Do đó, hệ thống hiện tại chưa phải là một mô hình end-to-end dự đoán trực tiếp giá bán sau khi xét hư hỏng từ một dataset thống nhất.

Vì chưa có nhãn ground truth cho giá xe sau khi xét đến từng loại hư hỏng cụ thể, dự án sử dụng một tầng điều chỉnh giá dựa trên luật ở bước cuối. Tầng này kết hợp base price từ mô hình hồi quy với các đặc trưng ảnh như số lượng hư hỏng, loại hư hỏng, diện tích vùng hỏng và mức độ nghiêm trọng để tạo ra giá tham khảo sau điều chỉnh. Cách tiếp cận này phù hợp với phạm vi của một dự án thực nghiệm, đồng thời vẫn thể hiện được ý tưởng kết hợp dữ liệu bảng và dữ liệu ảnh trong bài toán định giá xe cũ.

4. Phương pháp xây dựng mô hình

Trong dự án, hệ thống được xây dựng theo hướng pipeline nhiều mô-đun thay vì sử dụng một mô hình duy nhất. Cách tiếp cận này phù hợp với bài toán định giá xe ô tô cũ có xét đến tình trạng hư hỏng ngoại thất, vì dữ liệu đầu vào của hệ thống gồm cả dữ liệu bảng và dữ liệu ảnh. Mỗi loại dữ liệu có đặc điểm khác nhau và cần một mô hình xử lý riêng.

Cụ thể, dữ liệu bảng được sử dụng để dự đoán giá cơ bản của xe, dữ liệu ảnh được sử dụng để phát hiện các vùng hư hỏng và phân loại mức độ nghiêm trọng của hư hỏng. Kết quả từ các mô hình này sau đó được kết hợp trong một tầng điều chỉnh giá để đưa ra giá xe sau cùng.

4.1. Mô hình dự đoán giá từ dữ liệu bảng

Mô hình đầu tiên trong hệ thống là mô hình dự đoán giá cơ bản của xe từ dữ liệu dạng bảng. Dữ liệu đầu vào bao gồm các thông tin mô tả xe như hãng xe, tên xe, năm sản xuất, số km đã đi, loại nhiên liệu, hộp số, số chủ sở hữu, số chỗ ngồi, dung tích động cơ, công suất và mức tiêu hao nhiên liệu.

Sau bước tiền xử lý và feature engineering, dữ liệu được đưa vào các mô hình hồi quy để học mối quan hệ giữa đặc trưng của xe và giá bán. Trong dự án, các mô hình được thử nghiệm gồm Linear Regression, Random Forest

Regressor và XGBoost Regressor. Việc thử nghiệm nhiều mô hình giúp so sánh hiệu quả giữa các phương pháp tuyến tính và phi tuyến.

Linear Regression được sử dụng như một baseline đơn giản để đánh giá mức độ tuyến tính của dữ liệu. Random Forest có khả năng mô hình hóa các quan hệ phi tuyến và giảm overfitting thông qua việc kết hợp nhiều cây quyết định. XGBoost được lựa chọn là một trong các mô hình chính vì có khả năng học mạnh, xử lý tốt dữ liệu dạng bảng và thường đạt hiệu quả cao trong các bài toán regression.

Kết quả đầu ra của mô hình hồi quy là base price, tức là giá cơ bản của xe trước khi xét đến tình trạng hư hỏng ngoại thất. Giá trị này đóng vai trò làm nền tảng cho bước tính giá tham khảo sau điều chỉnh.

4.2. Mô hình phát hiện hư hỏng bằng YOLOv8

Mô hình thứ hai trong hệ thống là YOLOv8, được sử dụng cho bài toán phát hiện hư hỏng trên ảnh xe. Đây là bài toán object detection, trong đó mô hình cần xác định cả vị trí và loại hư hỏng xuất hiện trong ảnh.

Trong dự án, YOLOv8 được huấn luyện để phát hiện các loại hư hỏng ngoại thất như vết móp, vết xước, vết nứt, kính vỡ, đèn hỏng hoặc các dạng hư hỏng khác tùy theo class của dataset. Đầu ra của mô hình gồm tọa độ bounding box, nhãn hư hỏng và confidence score cho từng vùng phát hiện.

Dự án lựa chọn biến thể YOLOv8s vì đây là phiên bản cân bằng tốt giữa độ chính xác và tốc độ xử lý. So với YOLOv8n, YOLOv8s có khả năng học đặc trưng tốt hơn; trong khi so với YOLOv8m, mô hình nhẹ hơn và phù hợp hơn với việc tích hợp vào ứng dụng demo. Checkpoint yolov8_s100_640 được sử dụng làm mô hình phát hiện chính, với cấu hình YOLOv8s, 100 epoch và kích thước ảnh 640px.

Thông tin từ YOLO không chỉ được dùng để hiển thị bounding box trên giao diện, mà còn được sử dụng làm đầu vào cho các bước tiếp theo. Cụ thể, mỗi vùng hư hỏng được crop theo bounding box để đưa vào mô hình phân loại mức độ hư hỏng, đồng thời các thông tin như số lượng hư hỏng, loại hư hỏng và tỷ lệ diện tích vùng hỏng được dùng trong bước điều chỉnh giá.

4.3. Mô hình phân loại mức độ hư hỏng bằng ResNet18

Sau khi YOLO phát hiện các vùng hư hỏng, hệ thống sử dụng mô hình CNN với kiến trúc ResNet18 để phân loại mức độ nghiêm trọng của từng vùng hư hỏng. Bài toán này được xem là bài toán phân loại ảnh nhiều lớp, với các nhãn như minor, moderate và severe.

ResNet18 được lựa chọn vì đây là một kiến trúc CNN tương đối gọn, có khả năng học đặc trưng ảnh tốt và dễ tích hợp vào pipeline. Kiến trúc ResNet sử dụng residual connection, giúp quá trình huấn luyện ổn định hơn và hạn chế hiện tượng suy giảm hiệu năng khi mạng sâu hơn. So với các mô hình lớn hơn, ResNet18 có tốc độ suy luận nhanh hơn và phù hợp với quy mô của dự án.

Trong hệ thống, đầu vào của ResNet18 là ảnh vùng hư hỏng đã được crop từ kết quả bounding box của YOLO. Các ảnh này được resize và normalize theo chuẩn đầu vào của mô hình trước khi dự đoán. Đầu ra của mô hình là nhãn mức độ hư hỏng và điểm severity score tương ứng.

Kết quả phân loại mức độ hư hỏng giúp hệ thống đánh giá tình trạng ngoại thất chi tiết hơn. Ví dụ, cùng là một vết xước nhưng nếu mức độ nhẹ thì mức giảm giá thấp hơn, còn nếu mức độ nghiêm trọng thì mức giảm giá cần cao hơn. Vì vậy, mô hình CNN/ResNet18 đóng vai trò quan trọng trong việc hỗ trợ bước điều chỉnh giá cuối cùng của pipeline.

4.4. Cơ chế điều chỉnh giá sau hư hỏng

Sau khi có giá cơ bản từ mô hình hồi quy, kết quả phát hiện hư hỏng từ YOLO và mức độ nghiêm trọng từ ResNet18, hệ thống sử dụng một cơ chế điều chỉnh giá dựa trên luật để tạo ra mức giá tham khảo sau điều chỉnh.

Tầng điều chỉnh giá sử dụng các thông tin như:

- Số lượng vùng hư hỏng.
- Loại hư hỏng.
- Tỷ lệ diện tích vùng hỏng trên ảnh.
- Mức độ nghiêm trọng của từng hư hỏng.
- Điểm tổng hợp severity score.

Mỗi loại hư hỏng có thể được gán một trọng số khác nhau. Ví dụ, hư hỏng liên quan đến kính hoặc đèn có thể ảnh hưởng đến giá nhiều hơn một vết xước nhỏ. Tương tự, hư hỏng ở mức severe sẽ làm giảm giá nhiều hơn so với mức minor.

Do hiện tại dự án chưa có ground truth trực tiếp cho giá xe sau khi xét đến hư hỏng, tăng điều chỉnh giá chưa phải là một mô hình supervised learning. Thay vào đó, đây là một cơ chế rule-based adjustment dùng để minh họa cách kết hợp kết quả từ các mô hình vào quá trình ước lượng giá. Vì vậy, mức giảm giá trong phiên bản hiện tại mang tính tham khảo và có thể cần được hiệu chỉnh thêm khi có dữ liệu giao dịch thực tế.

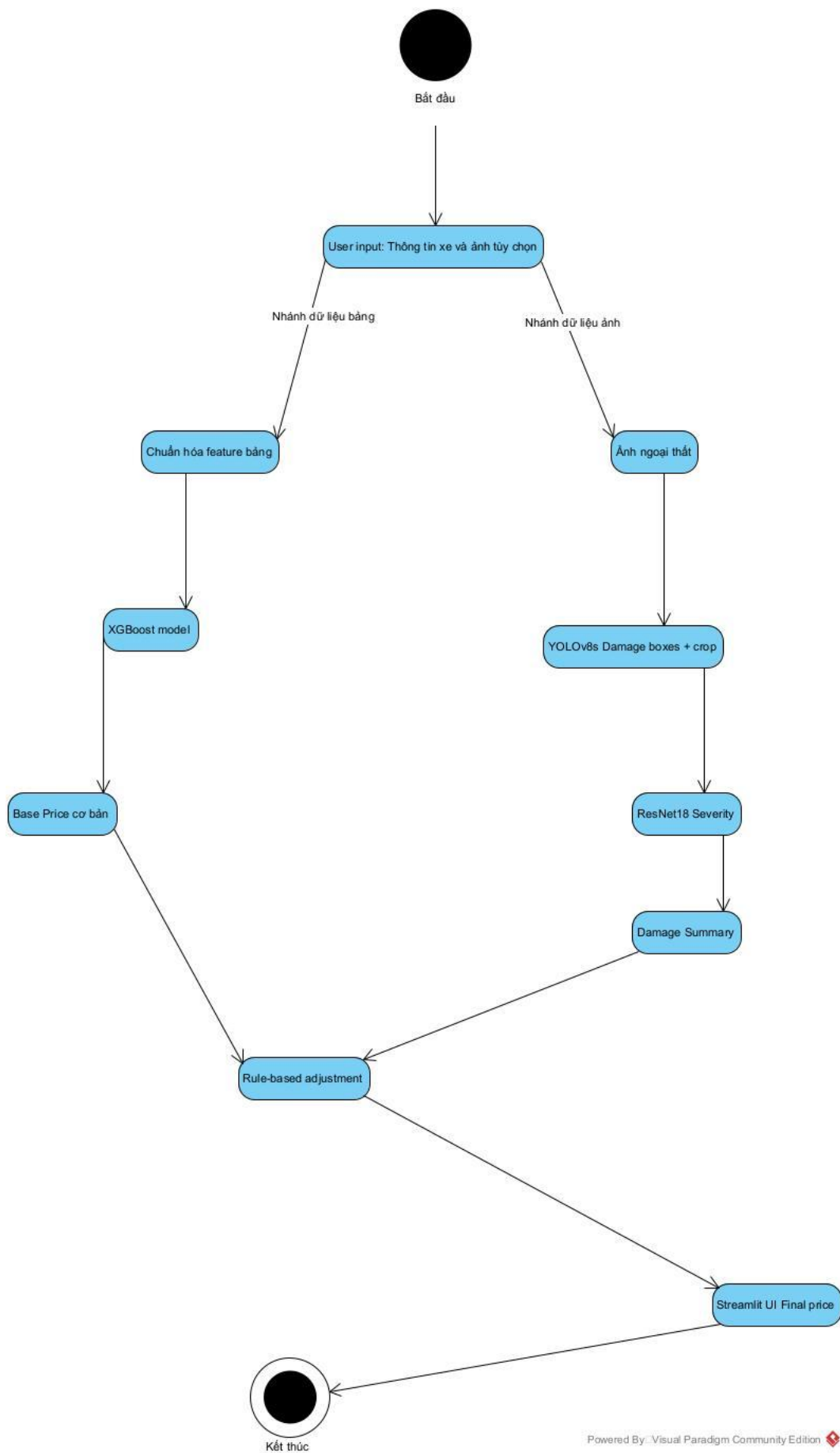
5. Thiết kế hệ thống

5.1. Kiến trúc tổng thể

Hệ thống được thiết kế theo kiến trúc pipeline nhiều mô-đun, trong đó mỗi mô-đun đảm nhiệm một nhiệm vụ riêng trong quá trình định giá xe ô tô cũ. Thay vì sử dụng một mô hình duy nhất cho toàn bộ bài toán, dự án tách hệ thống thành các thành phần nhỏ hơn gồm: mô-đun giao diện người dùng, mô-đun dự đoán giá cơ bản, mô-đun phát hiện hư hỏng, mô-đun phân loại mức độ hư hỏng và mô-đun điều chỉnh giá cuối cùng.

Cách thiết kế này phù hợp với bài toán của dự án vì dữ liệu đầu vào bao gồm cả dữ liệu dạng bảng và dữ liệu ảnh. Dữ liệu dạng bảng, bao gồm các thông tin như hãng xe, dòng xe, năm sản xuất, số km đã đi, loại nhiên liệu, hộp số, số chỗ ngồi, dung tích động cơ và công suất, được dùng để dự đoán giá cơ bản của xe. Trong khi đó, dữ liệu ảnh được dùng để phát hiện các vùng hư hỏng ngoại thất và đánh giá mức độ nghiêm trọng của từng hư hỏng.

Về tổng thể, hệ thống hoạt động theo luồng: người dùng nhập thông tin xe và có thể tải ảnh lên giao diện Streamlit. Thông tin xe luôn được đưa vào mô hình dự đoán giá cơ bản. Nếu người dùng có tải ảnh, ảnh xe tiếp tục được đưa vào mô hình YOLOv8 để phát hiện hư hỏng, sau đó các vùng hư hỏng được xử lý bởi CNN/ResNet18 để phân loại mức độ. Cuối cùng, hệ thống kết hợp giá cơ bản với thông tin hư hỏng để tính ra giá xe tham khảo sau điều chỉnh; nếu không có ảnh, giá tham khảo cuối cùng chính là giá cơ bản từ mô hình tabular.



Hình 5.1. Sơ đồ pipeline kiến trúc tổng thể của hệ thống

Kiến trúc này giúp hệ thống có tính mô-đun cao. Mỗi thành phần có thể được kiểm thử, thay thế hoặc cải thiện độc lập. Ví dụ, mô hình YOLO có thể được thay bằng checkpoint tốt hơn trong tương lai mà không cần thay đổi toàn bộ hệ thống. Tương tự, mô hình dự đoán giá hoặc công thức điều chỉnh giá cũng có thể được cải tiến riêng.

Mô tả kiến trúc tổng thể của hệ thống:

Đầu vào người dùng: Hiển thị rõ ràng các bước nhập liệu dữ liệu bảng và tải ảnh thông qua giao diện Streamlit.

Luồng xử lý dữ liệu song song:

- **Dữ liệu bảng:** Chuyển thẳng đến mô hình dự đoán giá cơ bản.
- **Dữ liệu hình ảnh:** Đi qua chuỗi xử lý phức tạp bao gồm YOLOv8 để phát hiện, cắt ảnh, và sau đó là ResNet18 để phân loại mức độ nghiêm trọng.

Phân bổ tổng hợp dữ liệu: Kết quả từ luồng hình ảnh (loại, số lượng, diện tích, độ nghiêm trọng) được tổng hợp lại.

Điều chỉnh giá Rule-based: Đây là nơi kết hợp giá cơ bản và dữ liệu hư hỏng để đưa ra mức điều chỉnh cuối cùng.

Hiển thị kết quả: Giao diện Streamlit hiển thị mức giá đã điều chỉnh và chi tiết các hư hỏng được phát hiện.

5.2. Luồng xử lý

Luồng xử lý của hệ thống bắt đầu từ việc người dùng truy cập giao diện ứng dụng. Tại đây, người dùng nhập các thông tin cơ bản của xe như hãng xe, dòng xe, năm sản xuất, số km đã đi, loại nhiên liệu, hộp số, số chủ sở hữu, số chỗ ngồi, dung tích động cơ, công suất và mức tiêu hao nhiên liệu. Người dùng có thể tải lên một hoặc nhiều ảnh chụp ngoại thất nếu muốn hệ thống phân tích thêm tình trạng hư hỏng.

Sau khi người dùng nhấn nút phân tích, hệ thống sẽ kiểm tra dữ liệu đầu vào và bắt đầu chạy pipeline dự đoán. Đầu tiên, thông tin dạng bảng của xe được chuẩn hóa về đúng định dạng feature mà mô hình tabular đã được huấn luyện. Các thông tin này sau đó được đưa vào mô hình hồi quy để dự đoán giá cơ bản

của xe. Giá cơ bản này phản ánh giá trị ước lượng của xe dựa trên các thuộc tính kỹ thuật, chưa xét đến tình trạng hư hỏng ngoại thất.

Nếu người dùng upload ảnh, các ảnh xe được đưa vào mô hình YOLOv8 để phát hiện các vùng hư hỏng. Với mỗi ảnh, mô hình YOLO trả về danh sách các bounding box, nhãn loại hư hỏng và độ tin cậy tương ứng. Hệ thống cũng tính thêm tỷ lệ diện tích vùng hư hỏng so với toàn bộ ảnh để phục vụ bước điều chỉnh giá.

Sau khi có kết quả phát hiện hư hỏng, hệ thống vẽ bounding box lên ảnh để tạo ảnh kết quả trực quan cho người dùng. Đồng thời, từng vùng hư hỏng được crop theo bounding box và đưa vào mô hình CNN/ResNet18 để phân loại mức độ nghiêm trọng. Mỗi vùng hư hỏng sẽ được gán một nhãn mức độ như minor, moderate hoặc severe, cùng với điểm severity score tương ứng.

Tiếp theo, hệ thống tổng hợp kết quả từ các vùng hư hỏng. Các thông tin được tổng hợp bao gồm tổng số hư hỏng, số lượng từng loại hư hỏng, mức độ nghiêm trọng cao nhất và điểm severity trung bình. Những thông tin này được kết hợp với giá cơ bản để tính toán mức giảm giá. Mức giảm giá được xác định dựa trên loại hư hỏng, diện tích vùng hỏng và mức độ nghiêm trọng.

Cuối cùng, hệ thống hiển thị kết quả lên giao diện, bao gồm giá cơ bản, số tiền giảm trừ và giá xe tham khảo sau điều chỉnh. Nếu có ảnh đầu vào, hệ thống hiển thị thêm ảnh gốc, ảnh có bounding box, bảng danh sách hư hỏng phát hiện được và bảng phân tích mức độ hư hỏng. Nếu không có ảnh, hệ thống bỏ qua nhánh YOLO/CNN và giá tham khảo cuối cùng bằng giá dự đoán từ XGBoost.

Tóm tắt luồng xử lý:

- Bước 1: Người dùng nhập thông tin xe và có thể tải ảnh xe lên hệ thống.
- Bước 2: Hệ thống chuẩn hóa thông tin xe thành các feature đầu vào.
- Bước 3: Mô hình tabular dự đoán giá cơ bản của xe.
- Bước 4: YOLOv8 phát hiện các vùng hư hỏng trên ảnh xe.
- Bước 5: Hệ thống vẽ bounding box và crop vùng hư hỏng.
- Bước 6: CNN/ResNet18 phân loại mức độ hư hỏng.
- Bước 7: Hệ thống tổng hợp thông tin damage và severity.
- Bước 8: Rule-based adjustment tính giá tham khảo sau điều chỉnh.
- Bước 9: Giao diện hiển thị kết quả cuối cùng cho người dùng.

5.3. Các module chính

Hệ thống được tổ chức thành nhiều module nhằm tách biệt rõ ràng giữa giao diện, xử lý mô hình, tiện ích và dữ liệu. Cách tổ chức này giúp mã nguồn dễ quản lý, dễ kiểm thử và dễ mở rộng.

Module đầu tiên là giao diện người dùng, được xây dựng bằng Streamlit trong file app.py và các file thuộc thư mục giao_dien/. Module này chịu trách nhiệm hiển thị form nhập thông tin xe, khu vực tải ảnh, ảnh xem trước, bảng kết quả phát hiện hư hỏng, kết quả phân loại mức độ và kết quả định giá. Đây là điểm tương tác chính giữa người dùng và hệ thống.

Module thứ hai là mô-đun định giá, nằm trong dịch_vu/dinh_gia.py. Module này chịu trách nhiệm dự đoán giá cơ bản từ mô hình tabular đã huấn luyện và tính toán giá tham khảo sau điều chỉnh. Đầu vào của module là thông tin xe đã được chuẩn hóa và kết quả hư hỏng từ các mô hình ảnh. Đầu ra là giá cơ bản, mức giảm trừ và giá tham khảo cuối cùng.

Module thứ ba là mô-đun phát hiện hư hỏng, nằm trong dịch_vu/phat_hien_hu_hong.py. Module này sử dụng mô hình YOLOv8 để phát hiện các vùng hư hỏng trên ảnh xe. Kết quả trả về gồm mã hư hỏng, tên ảnh, loại hư hỏng, độ tin cậy, tọa độ bounding box và tỷ lệ diện tích vùng hỏng. Module này cũng có chức năng vẽ bounding box lên ảnh để phục vụ hiển thị.

Module thứ tư là mô-đun phân loại mức độ hư hỏng, nằm trong dịch_vu/muc_do_hu_hong.py. Module này sử dụng mô hình CNN/ResNet18 để phân loại mức độ nghiêm trọng của từng vùng hư hỏng. Đầu vào là các crop ảnh theo bounding box từ YOLO, đầu ra là nhãn mức độ hư hỏng và điểm severity score. Module này cũng tổng hợp các kết quả severity để phục vụ phân hiển thị và điều chỉnh giá.

Module thứ năm là mô-đun tiện ích, nằm trong thư mục tien_ich/. Module này chứa các hàm hỗ trợ như định dạng tiền tệ, định dạng phần trăm, dữ liệu mẫu và quản lý trạng thái phiên làm việc của Streamlit. Việc tách các tiện ích này ra riêng giúp code chính ngắn gọn hơn và tránh lặp lại logic.

Module thứ sáu là mô-đun huấn luyện và xử lý dữ liệu, nằm trong thư mục src/. Thư mục này chứa các file phục vụ quá trình load dữ liệu, làm sạch dữ liệu, feature engineering, tiền xử lý, huấn luyện và đánh giá mô hình tabular. Đây là phần dùng trong giai đoạn phát triển mô hình, không trực tiếp hiển thị trên giao diện nhưng đóng vai trò quan trọng trong việc tạo ra model phục vụ hệ thống.

Các module chính trong dự án:

app.py

Điều phối luồng xử lý chính của ứng dụng Streamlit.

giao_dien/

Xây dựng các thành phần giao diện và hiển thị kết quả.

dich_vu/dinh_gia.py

Dự đoán giá cơ bản và tính giá tham khảo sau điều chỉnh.

dich_vu/phat_hien_hu_hong.py

Phát hiện vùng hư hỏng bằng YOLOv8 và vẽ bounding box.

dich_vu/muc_do_hu_hong.py

Phân loại mức độ hư hỏng bằng CNN/ResNet18.

tien_ich/

Chứa hàm định dạng, dữ liệu mẫu và quản lý session state.

src/

Chứa pipeline xử lý dữ liệu, huấn luyện và đánh giá mô hình.

Models/

Lưu các model đã huấn luyện như model tabular, YOLO và CNN.

Datasets/

Lưu dữ liệu dạng bảng và dữ liệu phục vụ huấn luyện.

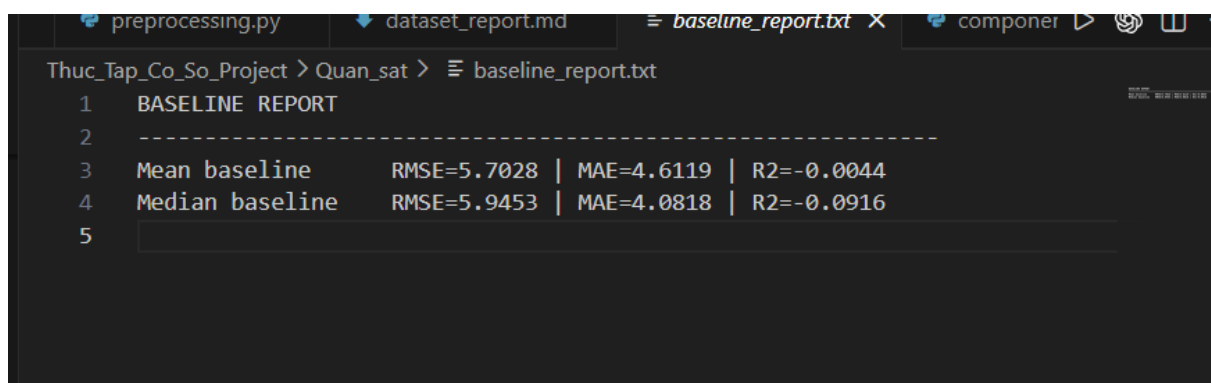
6. Kết quả thực nghiệm

6.1. Kết quả mô hình dự đoán giá từ dữ liệu bảng

Trong phần thực nghiệm với dữ liệu dạng bảng, dự án sử dụng tập dữ liệu xe ô tô cũ gồm 6019 dòng và 14 cột ban đầu. Sau quá trình làm sạch dữ liệu, loại bỏ các dòng không hợp lệ và xử lý các giá trị thiếu, tập train được chia thành 4780 mẫu huấn luyện và 1196 mẫu validation. Ngoài ra, tập test gồm 1223 mẫu được dùng để sinh dự đoán sau khi mô hình đã được huấn luyện.

Các đặc trưng đầu vào sau quá trình tiền xử lý gồm 11 đặc trưng số và 2 đặc trưng phân loại. Nhóm đặc trưng số bao gồm loại nhiên liệu, hộp số, số chủ sở hữu, mức tiêu hao nhiên liệu, dung tích động cơ, công suất, số chỗ ngồi, tuổi xe, số km trung bình mỗi năm, biển đánh dấu xe chạy nhiều và log số km đã đi. Hai đặc trưng phân loại là hãng xe và nhóm tên xe phổ biến.

Trước khi huấn luyện mô hình, dự án xây dựng baseline đơn giản bằng cách dự đoán theo giá trị trung bình và trung vị của tập train. Kết quả baseline như sau:



```
Thuc_Tap_Co_So_Project > Quan_sat > baseline_report.txt
1  BASELINE REPORT
2  -----
3  Mean baseline      RMSE=5.7028 | MAE=4.6119 | R2=-0.0044
4  Median baseline    RMSE=5.9453 | MAE=4.0818 | R2=-0.0916
5
```

Sau đó, dự án huấn luyện và so sánh ba mô hình hồi quy gồm Linear Regression, Random Forest và XGBoost. Kết quả trên tập validation như sau:

Mô hình	RMSE	MAE	R ²
Linear Regression	2.5325	1.8572	0.8019
Random Forest	1.6074	0.9962	0.9202
XGBoost	1.4313	0.8914	0.9367

Các giá trị RMSE và MAE được tính theo cùng đơn vị với biến mục tiêu trong dataset. Trong bộ dữ liệu xe cũ sử dụng cho dự án, giá xe được biểu diễn theo đơn vị lakh, vì

vậy $MAE = 0.8914$ có thể hiểu là sai số tuyệt đối trung bình khoảng 0.8914 lakh trên tập validation.

Kết quả cho thấy cả ba mô hình đều tốt hơn rõ rệt so với baseline. Trong đó, XGBoost đạt kết quả tốt nhất với $RMSE = 1.4313$, $MAE = 0.8914$ và $R^2 = 0.9367$. Điều này cho thấy mô hình XGBoost học tốt quan hệ phi tuyến giữa các đặc trưng của xe và giá bán.

Dự án cũng thực hiện kiểm tra overfitting bằng 5-fold cross-validation. Kết quả cho thấy XGBoost có R^2 trung bình CV đạt 0.9417, R^2 train đạt 0.9897 và khoảng cách train - CV là 0.0480. Theo tiêu chí đánh giá trong dự án, mô hình XGBoost chưa có dấu hiệu overfit rõ rệt và được chọn làm mô hình chính cho nhánh dự đoán giá cơ bản.

Used Car Price AI Demo

Tabular pricing, damage detection, severity grading, and price adjustment in one demo.

Input

Provide car details. Upload images if you want damage aware pricing.

Car Information Form

Brand

Toyota

Model

Innova 2.0 E

Year

2018

Number of Seats

5

KM Driven

45000

Fuel Type

Petrol

Transmission

Automatic

Owner Count

1

Fuel Consumption (km/l)

18.00

Engine Displacement (cc)

1500.00

Max Power

107.00

Upload car images (optional)

Drag and drop files here

Limit 200MB per file • PNG, JPG, JPEG

Browse Files

Upload images to see a preview.

Analyze Car

Pricing Results

Base Price

341,200,000 VND

Damage Deduction

0 VND

Final Adjusted Price

341,200,000 VND

Pricing Explanation

Top deduction reason: No detected damages

Total damage score: 0.0%

Trong trường hợp người dùng chỉ nhập thông tin xe và không tải ảnh hư hỏng lên hệ thống, ứng dụng hoạt động như một mô hình dự đoán giá baseline dựa hoàn toàn trên dữ liệu bảng. Với ví dụ xe Toyota Innova 2.0 E sản xuất năm 2018, đã đi 45.000 km, sử dụng xăng, hộp số tự động, 5 chỗ ngồi, dung tích 1500cc và công suất 107 mã lực, mô hình XGBoost dự đoán giá cơ bản của xe là **341.200.000 VND**. Do không có ảnh đầu vào nên hệ thống không thực hiện bước phát hiện hư hỏng bằng YOLO và không có thông tin phân loại mức độ hư hỏng bằng ResNet18. Vì vậy, mức giảm trừ do hư hỏng bằng **0 VND**, tổng điểm hư hỏng là **0.0%**, và giá cuối cùng sau điều chỉnh vẫn giữ nguyên là **341.200.000 VND**. Kết quả này minh họa vai trò của nhánh tabular baseline

trong hệ thống: cung cấp giá tham chiếu ban đầu của xe trước khi xét đến các yếu tố hư hỏng ngoại thất từ ảnh.

6.2. Kết quả thực nghiệm mô hình YOLO

6.2.1. Kết quả huấn luyện trên các cấu hình nền tảng

Mô hình nhận diện hư hỏng vỏ xe được thử nghiệm qua nhiều cấu hình kiến trúc của YOLOv8, bao gồm YOLOv8n, YOLOv8s và YOLOv8m. Sau giai đoạn benchmark ban đầu trên dataset khoảng 4000 ảnh, cấu hình YOLOv8s được giữ lại vì cân bằng tốt giữa tốc độ suy luận và độ chính xác. Từ baseline này, dự án tiếp tục huấn luyện phiên bản merge_data với cùng cấu hình nhưng mở rộng dữ liệu cho các lớp khó.

=== BẢNG SO SÁNH CÁC MÔ HÌNH ===

TÊN MODEL	PRECISION	RECALL	MAP50	MAP50-95
best_100_800sz.pt	0.7563	0.6772	0.7172	0.5494
best_m.pt	0.7724	0.6799	0.7217	0.5783
best_merge_data.pt	0.7272	0.6868	0.7016	0.5522
best_merge_data_1061.pt	0.7528	0.6573	0.6995	0.5491
best_n.pt	0.7630	0.6502	0.6988	0.5505
best_s_100.pt	0.7676	0.6554	0.7023	0.5431
best_s_70.pt	0.7312	0.6640	0.7010	0.5501
best_s_finetune.pt	0.7625	0.6841	0.7072	0.5567
best_v2.pt	0.7491	0.6850	0.7171	0.5620
best_89.pt	0.7267	0.6987	0.7158	0.5675

6.2.2. Tác động của dataset merge_data

Khác với các run cũ, mô hình YOLO chính hiện tại không còn là s_89 trên dataset gốc. Dự án giữ nguyên cấu hình YOLOv8s, epochs = 100, batch size = 16 và image size = 640, nhưng thay đổi dữ liệu huấn luyện bằng phiên bản merge_data. Dataset ban đầu có khoảng 4000 ảnh; sau đó, dự án bổ sung thêm 2307 ảnh cho ba lớp yếu nhất là crack, scratch và dent, nâng tổng số ảnh lên khoảng 6307 ảnh.

Điểm quan trọng của lần thử nghiệm này là dữ liệu mới không được thêm đều cho tất cả nhãn. Việc bổ sung tập trung vào các lớp có ranh giới thị giác khó, vùng hỏng nhỏ và dễ bị bỏ sót. Vì vậy, mục tiêu của merge_data không chỉ là tăng mAP tổng thể, mà còn cải thiện khả năng nhận diện các hư hỏng khó trong tình huống thực tế.

1. Run s_89 được giữ làm baseline tham khảo cho dataset gốc. Trong khi đó, mô hình merge_data được dùng làm mô hình chính vì có thêm dữ liệu học

cho các dạng hư hỏng mà hệ thống cần ưu tiên phát hiện như vết xước, vết nứt và vết móp.

2. Trong report mới tại thư mục Quan_sat/yolo_car_report, mô hình YOLOv8s trên merge_data đạt mAP@0.5 tốt nhất ở epoch 79, với Precision khoảng 0.8005, Recall khoảng 0.6820, mAP@0.5 khoảng 0.7256 và mAP@0.5:0.95 khoảng 0.5684. Nếu xét riêng mAP@0.5:0.95, epoch 75 đạt giá trị cao nhất khoảng 0.5719.

6.2.3. Quyết định lựa chọn mô hình cốt lõi merge_data

Việc lựa chọn merge_data dựa trên hai yếu tố chính. Thứ nhất, mô hình vẫn giữ được hiệu năng validation ổn định với cấu hình train giống baseline. Thứ hai, phiên bản dữ liệu mới phù hợp hơn với mục tiêu thực tế của dự án vì tăng số lượng mẫu cho các lớp khó nhất thay vì chỉ mở rộng dữ liệu một cách ngẫu nhiên.

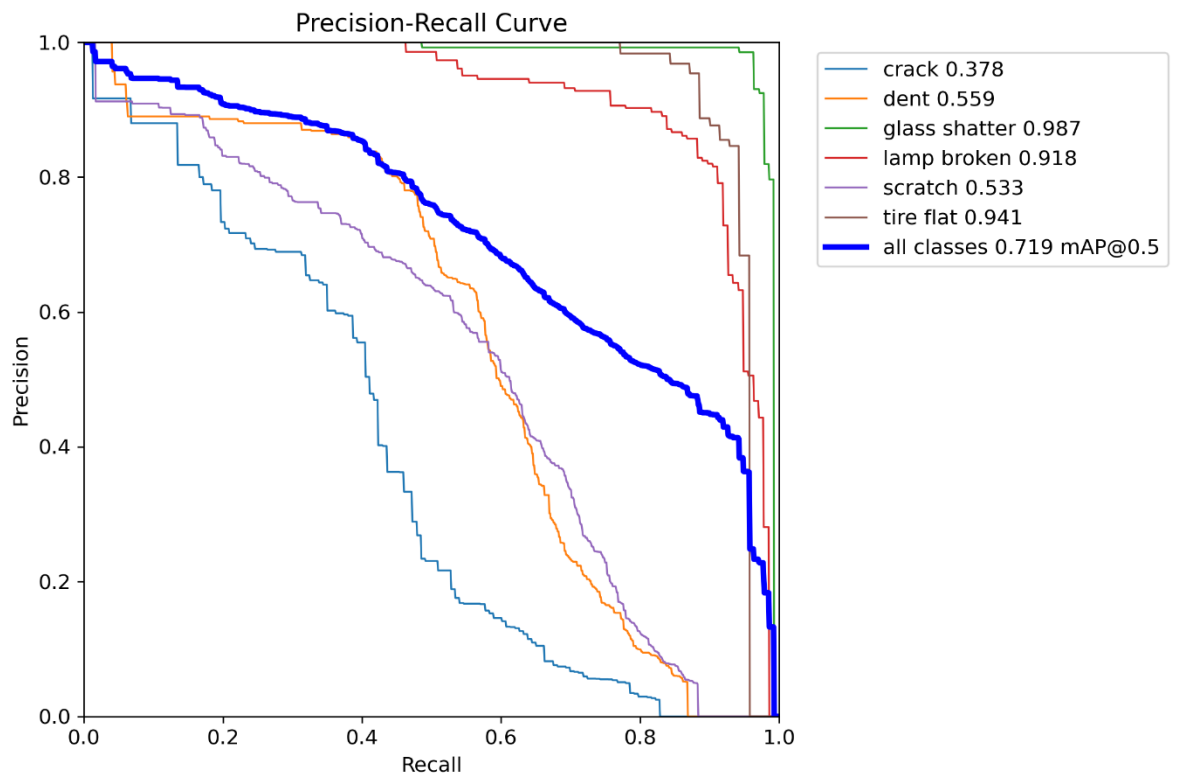
Ba phân lớp scratch, dent và crack ngoài thực tế có ranh giới thị giác mờ, kích thước nhỏ và dễ bị ảnh hưởng bởi ánh sáng, màu sơn hoặc góc chụp. Do đó, việc thêm 2307 ảnh cho riêng ba lớp này giúp mô hình có thêm ví dụ học đúng trọng tâm. Tuy nhiên, vì dữ liệu không được tăng đều cho mọi nhãn, khi phân tích kết quả cần xem xét cả metric tổng thể lẫn mục tiêu giảm bỏ sót các lớp hư hỏng khó.

Bên cạnh các chỉ số validation, khi đánh giá mô hình cũng cần xem xét chất lượng annotation của tập kiểm thử. Một nguyên nhân có thể ảnh hưởng đến metric là một số hư hỏng nhỏ như scratch, dent hoặc crack chưa được gán nhãn đầy đủ trong tập test. Trong trường hợp đó, một số dự đoán hợp lý của mô hình có thể bị hệ thống đánh giá tự động tính thành False Positive, làm giảm Precision hoặc mAP. Vì vậy, ngoài metric định lượng, dự án cũng quan sát thêm kết quả inference trực quan trên ảnh và video thực tế.

Thực nghiệm inference trên video và ảnh thực tế cho thấy mô hình YOLOv8s trên merge_data bám vết ổn định hơn, phát hiện tốt hơn các hư hỏng nhỏ thuộc nhóm scratch, dent và crack. Vì vậy, việc lựa chọn mô hình chính không chỉ dựa trên một con số mAP tĩnh, mà còn dựa trên khả năng tổng quát hóa trong tình huống sử dụng thực tế của hệ thống định giá xe.

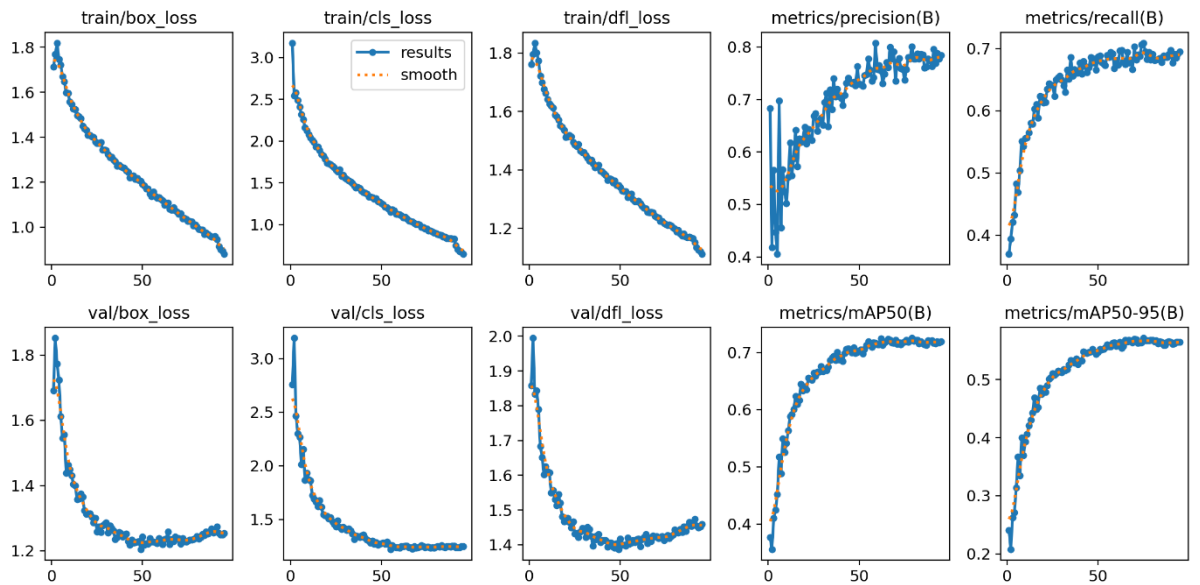
Vì mục tiêu của dự án là xây dựng hệ thống định giá có khả năng nhận diện hư hỏng ngoại thất trong thực tế, em quyết định lựa chọn trọng số tốt nhất best.pt của cấu hình YOLOv8s trên merge_data làm mô hình thị giác máy tính chính để tích hợp vào module định giá xe tổng thể.

Tham số	Giá trị
Model	YOLOv8s
Epochs	100
Batch size	16
Image size	640
Optimizer	auto
Pretrained	True
Dataset	coco_damage.yolov8 / merge_data
Patience	20



Trong report mới, epoch có mAP@0.5 cao nhất là epoch 79 với mAP@0.5 khoảng 0.7256. Ở các epoch cuối, mAP@0.5 vẫn dao động quanh 0.715-0.720 và mAP@0.5:0.95 quanh 0.562-0.565, cho thấy quá trình huấn luyện tương đối ổn định và không bị sụt giảm mạnh ở giai đoạn cuối.

Các chỉ số trong thư mục Quan_sat/yolo_car_report được lấy từ quá trình validation của YOLO. Nếu cần một con số test độc lập sau cùng, cần chạy đánh giá riêng bằng `model.val(split="test")` trên checkpoint đã chốt.



Kết quả của việc dự đoán so với nhãn thật



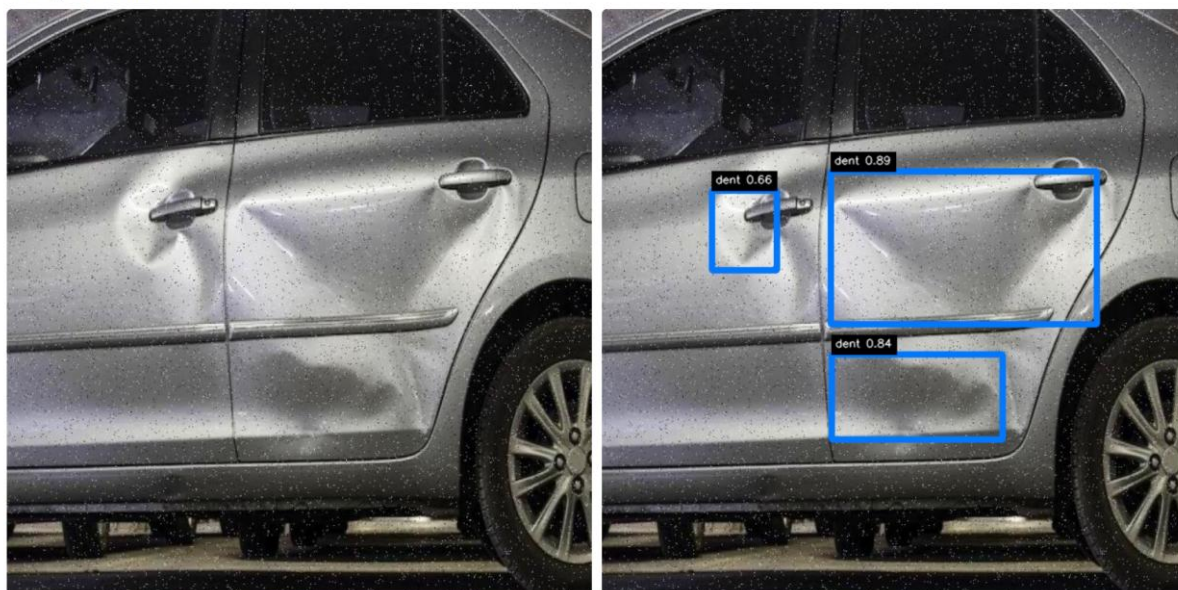
Kết quả thực nghiệm cho thấy YOLOv8s trên merge_data có khả năng phát hiện tương đối tốt các hư hỏng có hình dạng rõ ràng như kính vỡ, đèn hỏng

hoặc lớp xước, đồng thời được bổ sung thêm dữ liệu cho các lớp khó như vết nứt, vết móp và vết xước. Dù vậy, các lớp này vẫn là thách thức của bài toán vì vùng hổng nhỏ, dễ bị ảnh hưởng bởi ánh sáng, góc chụp, màu sơn xe và chất lượng annotation. Trong phạm vi dự án, checkpoint YOLOv8s merge_data được lựa chọn vì đạt mức cân bằng tốt giữa độ chính xác, tốc độ suy luận và khả năng tích hợp vào pipeline demo.

So sánh output của 2 mô hình YOLOv8s_89 và YOLOv8s_merge_data:

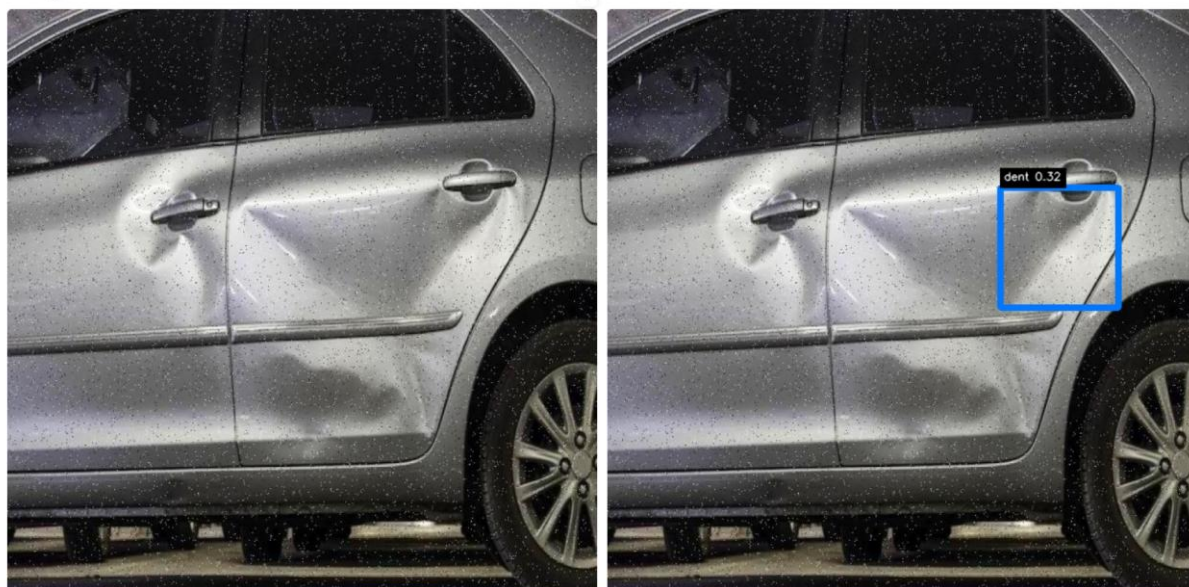
YOLOv8s_merge_data:

Damage Detection Viewer



YOLOv8s_89

Damage Detection Viewer



Quan sát kết quả trực quan cho thấy mô hình YOLOv8s trên merge_data

phát hiện được nhiều vùng hư hỏng hơn so với phiên bản YOLOv8s_89 trên dataset gốc, đặc biệt với các vết lõm (dent). Ở một số ảnh, mô hình cũ chỉ xác định được một vùng nhỏ, trong khi phiên bản merge_data cho kết quả bao quát hơn. Điều này phù hợp với mục tiêu bổ sung dữ liệu cho các lớp hư hỏng khó như dent, scratch và crack.

6.2.4. Nhận xét so sánh với Faster R-CNN

Bên cạnh YOLOv8, Faster R-CNN được xem xét như một hướng tham khảo thuộc nhóm two-stage detector. Khác với YOLO, Faster R-CNN thực hiện phát hiện đối tượng qua hai giai đoạn: đầu tiên sinh các vùng đề xuất có khả năng chứa đối tượng, sau đó phân loại và tinh chỉnh bounding box cho từng vùng. Cách tiếp cận này thường có ưu điểm về độ chính xác trong một số bài toán phát hiện đối tượng, đặc biệt khi cần định vị chi tiết.

Tuy nhiên, với mục tiêu của dự án là xây dựng một hệ thống demo có khả năng phát hiện hư hỏng xe và tích hợp vào pipeline định giá, YOLOv8 phù hợp hơn. YOLO là mô hình one-stage detector, có tốc độ suy luận nhanh hơn, dễ huấn luyện, dễ triển khai và được thư viện Ultralytics hỗ trợ tốt. Điều này giúp quá trình tích hợp vào ứng dụng Streamlit thuận tiện hơn so với Faster R-CNN.

Trong bài toán phát hiện hư hỏng xe, hệ thống không chỉ cần độ chính xác mà còn cần khả năng xử lý nhanh để người dùng có thể nhận kết quả gần như ngay sau khi tải ảnh lên. Faster R-CNN có thể cho kết quả tốt trong một số trường hợp, nhưng chi phí tính toán cao hơn và tốc độ suy luận thường chậm hơn. Vì vậy, mô hình này được xem như một hướng tham khảo để so sánh, trong khi YOLOv8s vẫn được chọn làm mô hình chính của hệ thống.

model	class_id_frnn	class_id_yolo_equiv	class_name	ap_50_95	ar_100
Faster R-CNN ResNet	1	0	crack	0.1204177886	0.3791410923
Faster R-CNN ResNet	2	1	dent	0.1190894246	0.3319105804
Faster R-CNN ResNet	3	2	glass shatter	0.06109387428	0.2283464521
Faster R-CNN ResNet	4	3	lamp broken	0.1040362641	0.2828358114
Faster R-CNN ResNet	5	4	scratch	0.1604667455	0.3674731255
Faster R-CNN ResNet	6	5	tire flat	0.2012279332	0.3757142723

6.3. Kết quả CNN

Đối với nhánh phân loại mức độ hư hỏng, dự án sử dụng mô hình CNN với kiến trúc ResNet18. Mô hình này nhận đầu vào là ảnh vùng hư hỏng và phân loại thành ba mức: minor, moderate và severe.

Quá trình huấn luyện được thực hiện qua hai giai đoạn. Ở giai đoạn đầu, mô hình đạt best validation accuracy khoảng 0.7856. Sau giai đoạn huấn luyện tiếp theo, best validation accuracy tăng lên 0.8155. Tuy nhiên, khi đánh giá checkpoint tốt nhất trên tập test, mô hình đạt accuracy khoảng 0.6590.

Chỉ số	Giá trị
Best validation accuracy giai đoạn 1	0.7856
Best validation accuracy giai đoạn 2	0.8155
Test accuracy	0.6590
Test loss	1.0933

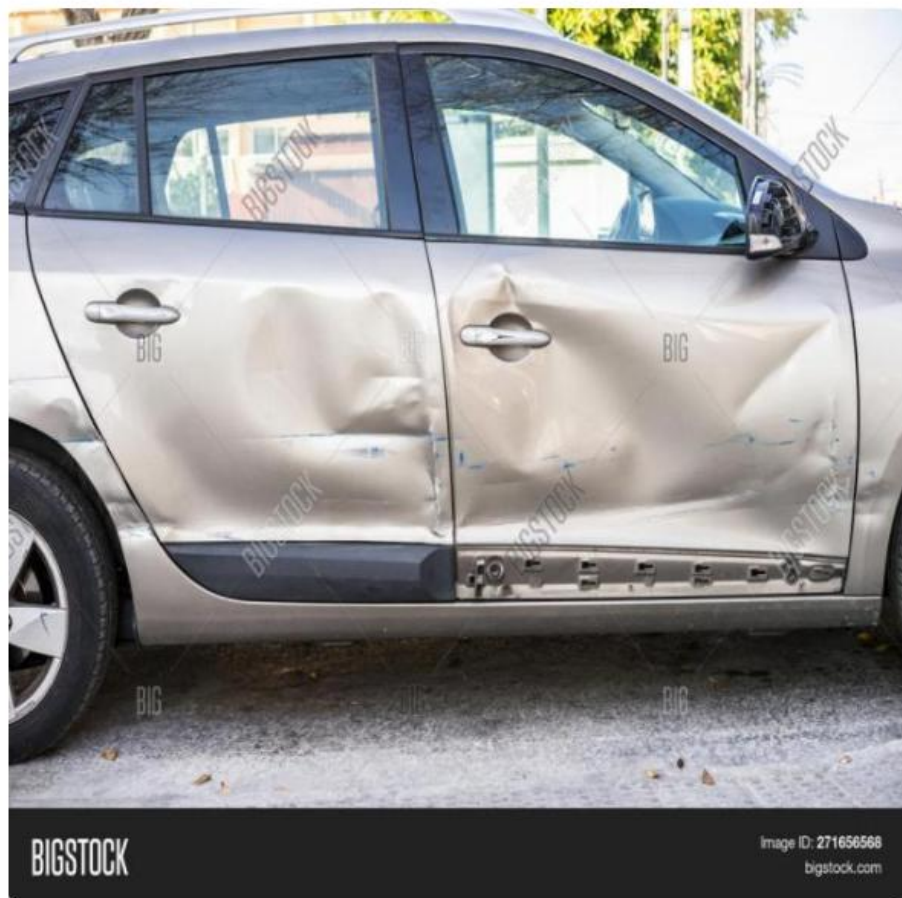
```
test_metrics = evaluate(model, test_loader, device)
print('Best checkpoint test metrics:', test_metrics)
print('Saved model:', model_path)
```

```
... Phase 2: fine-tune layer4 + fc
Epoch 01 | train_loss=0.6682 train_acc=0.7347 | val_loss=0.5270 val_acc=0.7992
Epoch 02 | train_loss=0.6006 train_acc=0.7565 | val_loss=0.5221 val_acc=0.8005
Epoch 03 | train_loss=0.5416 train_acc=0.7687 | val_loss=0.5319 val_acc=0.8114
Epoch 04 | train_loss=0.5249 train_acc=0.7859 | val_loss=0.5610 val_acc=0.7992
Epoch 05 | train_loss=0.4685 train_acc=0.8094 | val_loss=0.6014 val_acc=0.8005
Epoch 06 | train_loss=0.4564 train_acc=0.8189 | val_loss=0.5882 val_acc=0.8114
Epoch 07 | train_loss=0.4217 train_acc=0.8284 | val_loss=0.5443 val_acc=0.8100
Epoch 08 | train_loss=0.4128 train_acc=0.8289 | val_loss=0.5924 val_acc=0.7992
Epoch 09 | train_loss=0.3698 train_acc=0.8511 | val_loss=0.6403 val_acc=0.7897
Epoch 10 | train_loss=0.3691 train_acc=0.8538 | val_loss=0.6517 val_acc=0.8073
Epoch 11 | train_loss=0.3676 train_acc=0.8624 | val_loss=0.6640 val_acc=0.7788
Epoch 12 | train_loss=0.3586 train_acc=0.8597 | val_loss=0.6503 val_acc=0.8005
Epoch 13 | train_loss=0.3407 train_acc=0.8597 | val_loss=0.6445 val_acc=0.8033
Epoch 14 | train_loss=0.3238 train_acc=0.8728 | val_loss=0.6342 val_acc=0.8060
Epoch 15 | train_loss=0.3004 train_acc=0.8846 | val_loss=0.6700 val_acc=0.7978
Epoch 16 | train_loss=0.3093 train_acc=0.8773 | val_loss=0.6721 val_acc=0.7965
Epoch 17 | train_loss=0.2848 train_acc=0.8864 | val_loss=0.6729 val_acc=0.8087
Epoch 18 | train_loss=0.2956 train_acc=0.8841 | val_loss=0.7647 val_acc=0.8155
Epoch 19 | train_loss=0.2943 train_acc=0.8882 | val_loss=0.7890 val_acc=0.7815
Epoch 20 | train_loss=0.2862 train_acc=0.8904 | val_loss=0.6773 val_acc=0.7965
Epoch 21 | train_loss=0.2463 train_acc=0.9099 | val_loss=0.8100 val_acc=0.7788
Epoch 22 | train_loss=0.2673 train_acc=0.8932 | val_loss=0.7725 val_acc=0.8060
Epoch 23 | train_loss=0.2616 train_acc=0.9031 | val_loss=0.8799 val_acc=0.7856
Epoch 24 | train_loss=0.2521 train_acc=0.9072 | val_loss=0.8549 val_acc=0.7815
...
Epoch 50 | train_loss=0.1630 train_acc=0.9402 | val_loss=1.2788 val_acc=0.7775
Best val_acc: 0.8155
Best checkpoint test metrics: {'loss': 1.0932928847196774, 'accuracy': 0.658974358974359}
Saved model: /content/drive/MyDrive/yolov8_train_car/pkl/cnn_car.pkl
Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...
```

Kết quả này cho thấy mô hình ResNet18 đã học được đặc trưng phân biệt các mức độ hư hỏng trên tập validation, nhưng hiệu năng trên tập test còn thấp hơn đáng kể. Nguyên nhân có thể đến từ sự khác biệt phân phối giữa train/validation và test, số lượng dữ liệu chưa đủ lớn, hoặc ranh giới giữa các mức minor, moderate, severe chưa thật sự rõ ràng. Trong thực tế, việc phân biệt hư hỏng nhẹ và trung bình thường có tính chủ quan, nên mô hình dễ nhầm lẫn giữa các lớp gần nhau.

Đã chọn file: z1_b.jpg.7.EUYZUPWINZNIAN1C6WYL.jpg

Phan loại



Ảnh đầu vào

Kết quả: MODERATE (80.04%)

Xác suất theo từng nhóm

```
{
  "minor" : 18.8
  "moderate" : 80.04
  "severe" : 1.16
}
```

Hình trên minh họa kết quả demo của mô hình **CNN/ResNet18** trong bài toán phân loại mức độ hư hỏng xe. Ảnh đầu vào là phần thân xe bị móp rõ ở khu vực cửa và hông xe. Sau khi người dùng tải ảnh lên và nhấn nút phân loại, mô hình xử lý ảnh và dự đoán mức độ hư hỏng thuộc lớp **moderate** với xác suất **80.04%**. Bên cạnh nhãn dự đoán chính, hệ thống cũng hiển thị xác suất cho từng nhóm: **minor 18.8%**, **moderate 80.04%** và **severe 1.16%**.

Kết quả này cho thấy mô hình nhận diện hư hỏng ở mức trung bình, phù hợp với quan sát trực quan vì xe có vùng móp lớn nhưng chưa đến mức biến dạng nghiêm trọng toàn bộ cấu trúc. Phần hiển thị xác suất theo từng lớp giúp người dùng hiểu rõ hơn độ tự tin của mô hình, đồng thời hỗ trợ bước điều chỉnh giá sau này trong pipeline định giá xe cũ.

Tuy vậy, mô hình CNN vẫn có vai trò quan trọng trong pipeline vì nó bổ sung thông tin về mức độ nghiêm trọng của từng vùng hư hỏng. Nếu chỉ sử dụng YOLO, hệ thống chỉ biết vị trí và loại hư hỏng; khi kết hợp thêm CNN, hệ thống có thêm tín hiệu để ước lượng mức giảm giá. Trong phiên bản hiện tại, kết quả severity được xem như tín hiệu hỗ trợ cho tăng điều chỉnh giá tham khảo, chưa nên xem là đánh giá tuyệt đối về mức độ hư hỏng.

7. Đánh giá, hạn chế và hướng phát triển

7.1. Hạn chế

Mặc dù hệ thống đã xây dựng được pipeline hoàn chỉnh gồm dự đoán giá cơ bản, phát hiện hư hỏng, phân loại mức độ hư hỏng và điều chỉnh giá, dự án vẫn còn một số hạn chế nhất định.

Hạn chế đầu tiên là dữ liệu trong dự án chưa đồng bộ hoàn toàn theo từng chiếc xe. Dữ liệu bảng dùng để huấn luyện mô hình dự đoán giá xe được lấy từ một nguồn riêng, trong khi dữ liệu ảnh hư hỏng và dữ liệu phân loại mức độ hư hỏng lại đến từ các nguồn khác. Vì vậy, hệ thống hiện tại chưa học được trực tiếp mối quan hệ giữa ảnh hư hỏng của một chiếc xe cụ thể và giá bán thực tế của chính chiếc xe đó.

Hạn chế thứ hai là dự án chưa có ground truth cho giá xe sau khi xét đến hư hỏng. Mô hình tabular chỉ dự đoán giá cơ bản dựa trên các thông tin kỹ thuật của xe, còn phần điều chỉnh giá theo hư hỏng được xây dựng theo hướng rule-based. Điều này giúp hệ thống có thể hoạt động và giải thích được kết quả, nhưng mức giảm giá tham khảo vẫn phụ thuộc vào các quy tắc được thiết kế thủ công thay vì được học trực tiếp từ dữ liệu thực tế.

Hạn chế thứ ba nằm ở chất lượng và độ khó của dữ liệu ảnh hư hỏng. Một số lớp như scratch, crack và dent thường khó phát hiện hơn do vùng hư hỏng nhỏ, mảnh, dễ bị ảnh hưởng bởi ánh sáng, bóng đổ, màu sơn xe hoặc góc chụp. Điều này làm cho mô hình YOLO có thể bỏ sót hoặc nhầm lẫn giữa các loại hư hỏng có đặc điểm thị giác gần giống nhau.

Hạn chế thứ tư là mô hình CNN/ResNet18 phân loại mức độ hư hỏng vẫn còn dư địa cải thiện. Kết quả validation tương đối tốt nhưng kết quả trên tập test chưa thật sự cao, cho thấy mô hình có thể còn bị ảnh hưởng bởi sự khác biệt phân phối dữ liệu hoặc số lượng mẫu chưa đủ lớn. Ngoài ra, ranh giới giữa các mức minor, moderate và severe đôi khi mang tính chủ quan, khiến việc gán nhãn và học mô hình trở nên khó hơn.

Hạn chế thứ năm là hệ thống hiện mới dừng ở mức demo, chạy trên giao diện Streamlit và chưa được triển khai như một ứng dụng hoàn chỉnh cho người dùng thực tế. Các yếu tố như tốc độ xử lý trên nhiều ảnh lớn, khả năng lưu lịch sử định giá, quản lý người dùng, bảo mật dữ liệu và triển khai trên server chưa được xử lý đầy đủ.

7.2. Hướng phát triển

Trong tương lai, hướng phát triển quan trọng nhất là xây dựng một bộ dữ liệu đồng bộ hơn giữa thông tin xe, hình ảnh ngoại thất và giá bán thực tế. Nếu có thể thu thập dữ liệu gồm cùng một chiếc xe với đầy đủ thông tin kỹ thuật, ảnh hư hỏng và giá giao dịch sau khi xét tình trạng xe, hệ thống có thể tiến tới huấn luyện mô hình dự đoán giá cuối cùng một cách trực tiếp hơn thay vì chỉ dùng rule-based adjustment.

Một hướng phát triển tiếp theo là cải thiện chất lượng dữ liệu ảnh hư hỏng. Dự án có thể bổ sung thêm ảnh cho các lớp khó như scratch, crack và dent, đồng thời kiểm tra lại nhãn để giảm sai lệch annotation. Ngoài ra, có thể thêm các ảnh hard negative như phản sáng, vết bẩn, đường gân thân xe hoặc bóng đổ để giúp mô hình YOLO giảm nhầm lẫn trong quá trình phát hiện.

Đối với mô hình YOLO, có thể tiếp tục thử nghiệm các phiên bản mới hơn hoặc các cấu hình huấn luyện khác nhau. Ví dụ, dự án có thể so sánh thêm YOLOv8s, YOLOv8m hoặc các biến thể YOLO mới, thay đổi kích thước ảnh đầu vào, điều chỉnh augmentation, hoặc fine-tune từ checkpoint tốt nhất khi có

dữ liệu mới chất lượng hơn. Tuy nhiên, việc cải thiện dữ liệu vẫn nên được ưu tiên trước khi tăng độ phức tạp của mô hình.

Đối với mô hình CNN/ResNet18, có thể cải thiện bằng cách tăng số lượng ảnh huấn luyện, cân bằng lại số mẫu giữa các lớp minor, moderate và severe, áp dụng data augmentation hợp lý và đánh giá thêm bằng confusion matrix. Ngoài ResNet18, dự án cũng có thể thử các kiến trúc khác như ResNet50, EfficientNet hoặc MobileNet để so sánh giữa độ chính xác và tốc độ suy luận.

Phần điều chỉnh giá cũng có thể được cải tiến trong tương lai. Hiện tại, mức giảm giá được tính theo luật dựa trên loại hư hỏng, diện tích vùng hỏng và mức độ nghiêm trọng. Khi có dữ liệu thực tế hơn, có thể thay thế hoặc hiệu chỉnh tầng rule-based bằng một mô hình học máy, trong đó các đặc trưng từ YOLO và CNN được đưa vào cùng với dữ liệu bảng để dự đoán mức giảm giá hoặc giá tham khảo cuối cùng.

Về mặt hệ thống, ứng dụng có thể được mở rộng từ demo Streamlit thành một web application hoàn chỉnh hơn. Các chức năng có thể bổ sung gồm lưu lịch sử định giá, xuất báo cáo PDF, cho phép người dùng tải nhiều ảnh theo từng góc chụp, quản lý thông tin xe đã định giá và triển khai lên server để người dùng có thể truy cập từ xa.

Cuối cùng, nếu hướng đến ứng dụng thực tế tại Việt Nam, dự án cần bổ sung dữ liệu xe cũ từ thị trường Việt Nam. Dataset hiện tại chủ yếu phản ánh thị trường nước ngoài, nên giá trị dự đoán có thể chưa sát với giá xe tại Việt Nam. Việc thu thập dữ liệu từ các trang mua bán xe trong nước, kết hợp với tỷ giá, thương hiệu phổ biến và điều kiện sử dụng thực tế sẽ giúp hệ thống định giá phù hợp hơn với bối cảnh ứng dụng trong nước.

8. Kết luận

Trong dự án này, em đã xây dựng một hệ thống demo hỗ trợ dự đoán giá xe ô tô cũ có xét đến tình trạng hư hỏng ngoại thất bằng cách kết hợp Machine Learning và Computer Vision. Hệ thống không chỉ dựa trên các thông tin dạng bảng của xe như hãng xe, năm sản xuất, số km đã đi, loại nhiên liệu, hộp số, dung tích động cơ và công suất, mà còn khai thác thêm dữ liệu ảnh để phát hiện và đánh giá tình trạng hư hỏng thực tế của xe.

Về phần dự đoán giá cơ bản, dự án đã thực hiện các bước thu thập dữ liệu, làm sạch dữ liệu, xử lý giá trị thiếu, mã hóa biến phân loại, tạo đặc trưng mới và

huấn luyện các mô hình hồi quy như Linear Regression, Random Forest và XGBoost. Kết quả thực nghiệm cho thấy mô hình XGBoost đạt hiệu quả tốt nhất trong nhóm mô hình tabular, với khả năng học tốt mối quan hệ giữa các đặc trưng kỹ thuật của xe và giá bán.

Về phần xử lý ảnh, dự án đã tích hợp mô hình YOLOv8 để phát hiện các vùng hư hỏng ngoại thất trên ảnh xe. Mô hình YOLO giúp hệ thống xác định vị trí, loại hư hỏng và độ tin cậy của từng vùng phát hiện. Bên cạnh đó, mô hình CNN/ResNet18 được sử dụng để phân loại mức độ nghiêm trọng của từng vùng hư hỏng thành các mức như nhẹ, trung bình và nghiêm trọng. Kết quả từ hai mô hình ảnh này được dùng làm cơ sở cho bước điều chỉnh giá xe.

Một điểm quan trọng của dự án là việc thiết kế pipeline nhiều mô-đun, trong đó mỗi thành phần đảm nhiệm một nhiệm vụ riêng: mô hình tabular dự đoán giá cơ bản, YOLO phát hiện hư hỏng, CNN phân loại mức độ hư hỏng và tầng rule-based adjustment tính toán giá tham khảo sau điều chỉnh. Cách tiếp cận này giúp hệ thống dễ hiểu, dễ mở rộng và phù hợp với phạm vi của một dự án thực nghiệm.

Thông qua giao diện demo bằng Streamlit, người dùng có thể nhập thông tin xe, tải ảnh xe lên và nhận kết quả gồm giá cơ bản, danh sách hư hỏng phát hiện được, mức độ hư hỏng và giá xe tham khảo sau điều chỉnh. Điều này giúp dự án không chỉ dừng lại ở bước huấn luyện mô hình mà còn tiến tới một ứng dụng có khả năng minh họa trực quan quy trình định giá xe cũ.

Tuy nhiên, dự án vẫn còn một số hạn chế. Các bộ dữ liệu sử dụng trong hệ thống chưa đồng bộ hoàn toàn theo từng chiếc xe, chưa có nhãn giá bán sau khi xét đến hư hỏng, và phần điều chỉnh giá hiện vẫn dựa trên luật thay vì được học trực tiếp từ dữ liệu thực tế. Ngoài ra, mô hình phát hiện và phân loại hư hỏng vẫn có thể được cải thiện thêm, đặc biệt với các lớp khó như vết xước, vết nứt và vết móp.

Nhìn chung, dự án đã hoàn thành được mục tiêu xây dựng một hệ thống thử nghiệm kết hợp dữ liệu bảng và dữ liệu ảnh trong bài toán định giá xe ô tô cũ. Qua quá trình thực hiện, em đã có cơ hội vận dụng các kiến thức về xử lý dữ liệu, Machine Learning, Deep Learning, Computer Vision và xây dựng ứng dụng demo. Đây là nền tảng quan trọng để tiếp tục phát triển hệ thống trong tương lai, đặc biệt theo hướng bổ sung dữ liệu thực tế tại thị trường Việt Nam, cải thiện mô hình hư hỏng và xây dựng cơ chế định giá sát thực tế hơn.

9. Tài liệu tham khảo

Pytorch và python:

- https://www.learnpytorch.io/03_pytorch_computer_vision/
- <https://www.w3schools.com/>

Video, tài liệu về xgboost: <https://www.youtube.com/watch?v=a4jzMCXkQ0U>

Tài liệu, tìm hiểu về Decision Tree:

<https://machinelearningcoban.com/2018/01/14/id3/>

CNN và Resnet18:

- https://vi.d2l.ai/chapter_convolutional-modern/index.html
- <https://cnn-playground.live/>

YOLO và các phiên bản: <https://docs.ultralytics.com/models/yolov8>

Metrics:

- <https://docs.ultralytics.com/reference/utils/metrics>

Faster-RCNN:

- <https://www.ultralytics.com/blog/what-is-r-cnn-a-quick-overview>
- <https://minhtrieu.vn/thi-giac-may-machine-vision/phat-hien-doi-tuong-voi-yolo-faster-r-cnn/>

Finetune và Transfer learning:

https://docs.pytorch.org/tutorials/beginner/transfer_learning_tutorial.html

Object detection: <https://docs.ultralytics.com/tasks/detect>